AOSP 进阶丛书 · 第十五册 · ANDROID 13 (API 33)

构建系统 Soong 实战

Soong 四段管线（Android.bp→ninja→编译→镜像）、device 树三层继承与 product 配置、构建变体（eng/userdebug/user）、Soong 模块类型速查、out/ 与增量优化实战。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：系统的"工厂" 2
第一章 构建管线：bp→ninja→镜像 2
第二章 device 树与 product 配置 4
第三章 Soong 模块类型速查 5
第四章 out/ 与增量优化 5
第五章 设计决策六问 6
附录 A 实战场景 6
附录 B 快问快答（12 条） 7
附录 C 自检题 7
第六章 soong_ui 主循环：阶段机与 combined.ninja 8
第七章 Android.bp → build.ninja：soong_build 的两次遍历 13
第八章 product 配置继承图：inherit-product 与 rbc 双轨 18
第九章 Kati 边界与打包签名：从 .mk 到 OTA 包 23
第十章 APEX 构建：从 apex{} 到 Mainline 交付物 28
第十一章 签名体系与 AVB：testkey、releasekey 与 vbmeta 31
第十二章 遗留 Android.mk 的生存边界与 androidmk 35
第十三章 构建性能工程：并发、内存池与 RBE 38
教科书习题集（分级） 42
术语表 45
排障手册：构建系统实战问题定位 46
构建系统核心速查表 49

本册速览

Soong 构建管线：Android.bp→ninja→image
device tree 继承链与 product 配置（lunch 体系）
构建变体 eng/userdebug/user 差异
Soong 模块类型速查（cc_binary/java_library/aidl_interface/apex…）
out/ 产物结构与增量构建实践

前言：系统的"工厂"

设计洞察 · 学习目标

1. 能把"工厂"比喻映射到四个生产环节（原料=源码、产线=阶段机、工位=模块、成品=镜像）；2. 能列出本册交付的四项能力（答机制、追调用链、定位问题、面试作答）；3. 能说明本册与第1册"构建7步教程"的分工——那边教操作，这边讲机理。

源码是原料，构建系统是**工厂**——本册教你读懂这座工厂：Soong 怎么读 Android.bp、lunch 怎么选产品、out/ 里产出了什么、增量怎么加速。读完你能回答"改一行 framework 怎么最快上机""为什么 lunch 选错了全编失败""out/ 占了 200GB 怎么清"这类实际问题。

第一章 构建管线：bp→ninja→镜像

设计洞察 · 学习目标

1. 能背出 bp→ninja→编译→镜像四段的输入/输出与代表文件；2. 能按场景选用 m/m services/m snod/m nothing 并估算耗时；3. 能说出 Soong Go 框架展开变体的四条轴（arch×os×link×APEX）。



图 1 Soong 四段：Android.bp→依赖图（ninja）→编译（clang/javac）→镜像打包（Makefile），常用命令族与 out/ 结构

1.1 四段逐段

text 构建管线（锚点+讲解注释）

```
① 描述层：Android.bp（声明式）+ Android.mk（Kati 遗留）
// build/soong/android/（Go 框架）解析 bp→ModuleContext
// 变体轴：arch(arm64/x86_64) × os(android/vendor) × link(shared/static) × APEX
② 依赖图：Soong→build.ninja（主目标文件）
// ninja 是"最小构建系统"——只管执行不管解析
// 一个 cc_binary 可能展开 90+ 目标（变体组合爆炸）
③ 编译：ninja -j N 调 clang/javac/dex2oat
// 增量：out/soong/.intermediates/ 按模块名+输入哈希缓存
// m services 只重编受影响模块链（30s-5min vs 全编 2-4h）
④ 镜像：build/make/core/Makefile（Kati 侧）
// system.img(ext4/erofs) · boot.img(mkbootimg) · *.apex · vbmeta(AVB)
// device/<oem>/<product>/BoardConfig.mk 决定尺寸/签名/分区
```

1.2 实用命令族

命令	用途	耗时参考
source build/envsetup.sh	加载函数	秒
lunch aosp_x86_64-userdebug	选产品	秒（首次需交互）
m	全编	2-4h（首次）
m services	只编 services.jar	30s-2min
m snod	只重打 system.img	~1min
mm	当前目录	秒-分钟
adb root; adb sync	推增量	秒
m nothing	验证语法	秒（不产出）
m module-info	依赖图 dump	分钟

1.3 Soong 框架逐段注释（Go 侧怎么展开变体）

text build/soong/android/module.go（结构+讲解注释）

```
// 讲解注释①：每个 Android.bp 模块类型对应一个 Go 结构体：
// cc_binary → AndroidModulesEntry{name:"cc_binary", ...}
// Soong 解析 bp 文件时反射调用工厂函数创建模块对象
// 讲解注释②：模块的 Variants() 方法返回所有需要展开的变体轴：
// [arm64-android, arm-android, x86_64-android, ...]
// × [vendor, ] × [shared, static] × [apex1, apex2, ...]
// → 每个"变体组合"生成一个独立的编译目标
// 讲解注释③：依赖解析（deps.go）：
// shared_libs: ["libbinder"] → 找到 libbinder 模块 →
// 按当前变体匹配 libbinder 的对应变体 → 生成 DT_NEEDED
// → ninja 文件记录依赖关系（改了 libbinder → 重链所有下游）
// 讲解注释④：产物（outputs）：
// 编译产物路径规则：out/soong/.intermediates/<path>/<variant>/<output>
```

```
// 每个模块独立目录——增量按输入哈希判断是否需要重编
```

先修要求与学习路线

先修：已读第一册（架构总览）。

读完本书你将能够：

- 解释构建系统实战的核心机制与设计决策
- 追踪关键调用链并标注每次跨进程跳
- 使用 dumpsys/日志定位本领域问题
- 回答书7 对应章节的面试题

> **设计洞察 · 逐段精读：为什么 Go 而不是 Python 或 C++**
> 编译速度（Go 编译到内存→秒级解析数万 bp）+ 类型安全（struct/interface 模型匹配"声明式 bp→强类型模块"）+ 工具链统一（Go test/Go doc）。这是"工程语言选型服务于工程目标"的教科书案例。

第二章 device 树与 product 配置

> **设计洞察 · 学习目标**
> 1. 能画出 chipset→device→product 三层继承的变量叠加模型；2. 能区分 AndroidProducts.mk/device.mk/BoardConfig.mk 三文件的职责；3. 能解释 eng/userdebug/user 三变体在 root、ro.debuggable、SELinux 模式上的差异。

`device/<oem>/<product>/` 是产品的"身份证"。三层继承：**chipset → device → product**（每层 device.mk 叠加 PRODUCT_PACKAGES 等变量）。lunch 的目标名由这里的 `AndroidProducts.mk` 声明。

text device 树结构（以 device/google/cuttlefish 为参考）

```
AndroidProducts.mk // 产品列表（lunch 可见的目标名）
device.mk // 本设备的包+配置+overlay
BoardConfig.mk // 板级：分区尺寸/签名/内核路径
sepolicy/ // 设备特有 SELinux 策略
rro/ // Runtime Resource Overlay（书1 §六.2）
```

2.1 构建变体

变体	用途	关键差异
eng	开发	root 可用/调试开/SELinux permissive
userdebug	调试发布	adb root 可用/SELinux enforcing
user	正式	无 root/全限制（量产唯一合法）

面试高频："userdebug 与 user 的差异不止 root——还有 ro.debuggable（影响 logcat/属性写）与 SELinux 模式"。

第三章 Soong 模块类型速查

设计洞察 · 学习目标

1. 能按产出物归类常用模块类型（可执行/库/JAR/APK/APEX/测试）；2. 能为 HAL 接口、系统 App、编译期生成文件三类需求选出正确模块类型；3. 能说出 aidl_interface 自动展开的四后端（java/cpp/ndk/rust）。

模块类型	产出	典型用例
cc_binary	可执行	init/servicemanger/你的 HAL 服务
cc_library_shared	.so	libbinder/libutils
java_library	.jar	framework.jar/services.jar
android_app	APK	SystemUI/Launcher3
aidl_interface	四后端绑定	HAL 接口/系统服务接口
apex	APEX 模块	com.android.art/tethering
android_test	测试 APK	CTS 用例
genrule	生成文件	编译期代码/资源生成
filegroup	文件集	跨模块引用源码集
sdk_snapshot	SDK 库	Mainline 模块对 SDK 的固定引用

设计洞察 · 选型口诀

C/C++ 用 cc_*；Java 用 java_library 或 android_app；接口用 aidl_interface（自动四后端）；可独立升级用 apex；其他类型查 [Soong Modules Reference](https://source.android.com/docs/setup/build)。

第四章 out/ 与增量优化

设计洞察 · 学习目标

1. 能默画 out/ 四大子目录及各自内容；2. 能解释模块输入哈希增量机制与"改 framework 一行"的最快上机路径；3. 能在空间紧张时按层级选择 installclean/m clean/rm -rf out。

text out/ 结构速查

```
out/target/product/<device>/ → 镜像（system.img/boot.img/…）
out/soong/.intermediates/<path>/ → 模块中间产物（.o/.jar/.dex）
```

```
out/host/linux-x86/bin/ → host 工具（adb/fastboot/soong_ui）
out/visible_modules/ → 模块可见性声明缓存
```

增量原理：Soong 为每个模块计算输入哈希；`m services` 只重编哈希变化的模块及下游依赖——编辑 framework/java → services.jar → system.img → adb sync，最快路径。**空间回收**：`m clean`（删 out/ 下构建产物，保留 .zip）；`rm -rf out`（彻底重来，首次全编 2-4h）。

第五章 设计决策六问

> **设计洞察 · 学习目标**
> 1. 能复述六个设计决策的"为什么"并各举一个反例场景；2. 能用变体爆炸、Kati 存废、Bazel 试验三条线索评估构建系统的演进方向；3. 能把决策理由映射到面试题的答法（书7 对应章节）。

- **为什么声明式 bp 取代命令式 mk？** 变体由框架统一展开（mk 要手写 ifeq 分支）；声明式可静态分析校验（host_configverification）。
- **为什么 Kati 保留？** 数百万行 Make 规则不可能一夜迁完——androidmk 工具可自动转换，两者长期共存。
- **为什么 out/ 那么大？** 每个变体一套中间产物（90+ 目标×数万模块）；`m nothing` 验证后可 `rm out/soong/.intermediates` 只留缓存骨架。
- **为什么 Bazel 还在试验？** Soong 是 Google 内部产物闭源考量+Bazel 的 remote execution 更优——未来可能三引擎共存（bp+kati+bazel）。
- **为什么 device 树有 BoardConfig 而不用 bp？** 板级配置（分区尺寸/签名密钥）是"硬件事实"而非"构建逻辑"——mk 的简单变量足够且更稳定。
- **为什么 lunch 是交互式的？** 产品目标可能几十个（chipset×device×variant 组合），交互式选择比命令行全名更友好——`lunch` 不带参数列出全部。

附录 A 实战场景

> **设计洞察 · 学习目标**
> 1. 能按"改什么→编什么→怎么上机"三问速查本表选命令；2. 能预估每条命令的量级耗时并据此排工作流；3. 能区分"重编模块"与"重打镜像"两类操作适用场景。

场景	命令	耗时	
改 framework/base 一行	m services && adb sync	<2min	
加新系统 App	m MyApp && adb sync	<1min	
改 SELinux 策略	m system/sepolicy && adb sync	<1min	

场景	命令	耗时	
新 HAL 五步验证	m android.hardware.mysvc && flash vendor	首次+5min	
验证 bp 语法	m nothing	秒	
查模块依赖	m module-info	grep mymod	分钟
清理重编	m clean && m	2-4h	

附录 B 快问快答（12 条）

设计洞察 · 学习目标

1. 用 12 个一句话问答自检第六至九章核心机制的记忆牢固度；2. 能对答不出的条目回跳对应章节精读；3. 能把问答式记忆转化为面试口头作答。

- Soong 的输入输出？——输入 Android.bp→输出 build.ninja。
- 一个 cc_binary 可能展开多少目标？——90+（arch×os×link×APEX 变体爆炸）。
- 增量靠什么？——out/soong 的模块输入哈希缓存。
- 最快验证语法？——m nothing。
- lunch 什么格式？——product-variant（如 aosp_x86_64-userdebug）。
- BoardConfig 管什么？——分区尺寸/签名密钥/内核路径。
- system.img 谁打的？——build/make/core/Makefile（Kati 侧）。
- APEX 构建用什么模块类型？——apex{}。
- AIDL 接口定义用什么？——aidl_interface{}（自动四后端）。
- userdebug 与 user 差异？——root+ro.debuggable+SELinux 模式。
- out/ 可删吗？——rm -rf out 完全重建（首次 2-4h）。
- device 树的继承链？——chipset→device→product（三层叠加）。

附录 C 自检题

设计洞察 · 学习目标

1. 用四道开放题检验能否脱离书本复述构建系统的骨架（管线/命令/增量/职责边界）；2. 能把自检失分点映射回第一至五章对应小节；3. 能对照教科书习题集（分级）的参考答案要点校准作答深度。

- 画 Soong 四段管线，标出 ninja/Makefile 各管哪段。
- 改一行 ActivityManagerService.java 到手机上生效的最短路径。
- 为什么 m services 比 m 快？增量机制的本质。

- BoardConfig.mk 与 Android.bp 各自的职责边界。

第六章 soong_ui 主循环：阶段机与 combined.ninja

> **设计洞察 · 学习目标**
> 1. 能画出 m 命令从 envsetup.sh 到 ninja 执行的五阶段流水线，标出每阶段入口函数与产物文件；2. 能解释 combined-<product>-<variant>.ninja 的 subninja 并线结构与 highmem_pool 的并发语义；3. 能说明切 lunch 触发自动 installclean 的判定文件、代码位置与逃生开关；4. 能用阶段位掩码解释 m nothing/m installclean/m clean 为什么快。

6.1 链路位置：四段管线背后的"调度器"

第一章 `::fig build-pipeline` 画出了 bp→ninja→镜像 的四段流水线；本章补上那位隐形指挥——**soong_ui**。`m` 不是 make：envsetup.sh 定义的 m 函数最终调起 `build/soong/soong_ui.bash`，它用 Microfactory 自举出 Go 写的 soong_ui 二进制，再由 soong_ui 按阶段调度 ckati、soong_build、ninja——它自己从不解析任何 .mk 或 .bp。四段管线与阶段位的对应关系：RunProductConfig（选产品出变量）、RunSoong（bp→build.ninja）、RunKati（mk→build-<product>.ninja）、RunNinja（真正编译打包）。



图 2 soong_ui 阶段机：五阶段流水线、两份 ninja 清单的分工与增量构建的决定因素

soong 自举过程（build/soong/ui/build/soong.go:60-80 的原始注释）分三步：

- 1. 用 Microfactory（极简 Go 构建工具，只认文件路径不解析 bp）编出 soong_ui 自身；

- 2. 简化版 soong_build 读描述自身构建的 Blueprint 文件，产出 .bootstrap/build.ninja；
- 3. soong_ui 执行 .bootstrap/build.ninja 编出完整版 soong_build，它再去解析全树 Android.bp。

> **设计洞察 · 逐段精读：自举为什么用 Microfactory**
> 鸡生蛋问题——soong_build 解析 Android.bp 的能力是它自己编出来的。Microfactory 只需几百行代码，绕过 bp 解析、直接按 Go 包路径编译，先造出"能编 soong_build 的 soong_build"再进入正循环；这与 ART 生态 dex2oat 的自举同构：把"编译器如何被编译"显式建进构建图，而不是藏在脚本里。

6.2 Build()：唯一的主循环

soong_ui 的全部行为集中在 build/soong/ui/build/build.go 的 Build() 函数（188 行起）。先看阶段位定义（94-108 行），再走读主循环：

go build/soong/ui/build/build.go:94-108（原文+讲解注释）

```
const (
_ = iota
// Whether to run the kati config step.
RunProductConfig = 1 << iota // 讲解注释①：iota 从 1 起算——RunProductConfig=2，不是 1
// Whether to run soong to generate a ninja file.
RunSoong = 1 << iota // 讲解注释②：=4
// Whether to run kati to generate a ninja file.
RunKati = 1 << iota // 讲解注释③：=8
// Whether to include the kati-generated ninja file in the combined ninja.
RunKatiNinja = 1 << iota // 讲解注释④：=16——"是否把 kati 产物并进 combined"
// Whether to run ninja on the combined ninja.
RunNinja = 1 << iota // 讲解注释⑤：=32
RunBazel = 1 << iota // =64，mixed build 用
RunBuildTests = 1 << iota // =128，附带悬空规则检查
RunAll = RunProductConfig | RunSoong | RunKati | RunKatiNinja | RunNinja
RunAllWithBazel = RunProductConfig | RunSoong | RunKati | RunKatiNinja | RunBazel
)
```

主循环就是"按位分流"——每个阶段一个 if，产物落在 out/ 的固定位置：

go build/soong/ui/build/build.go:188-256（骨架+讲解注释）

```
func Build(ctx Context, config Config) {
ctx.Verboseln("Starting build with args:", config.Arguments())
// 讲解注释①：Arguments() 里的"services"等字串是构建目标（goals），
// 最终原样透传给最后的 ninja；help/clean 等特例在下面提前分流。
buildLock := BecomeSingletonOrFail(ctx, config) // 讲解注释②：同一 out/ 独占锁——两个终端同时 m 会直接报错
defer buildLock.Unlock()
// 讲解注释③：m clean/clobber 在此提前 return（不走阶段机），代码同下方注释语义
if inList("clean", config.Arguments()) ||
inList("clobber", config.Arguments()) {
clean(ctx, config)
return
}
defer waitForDist(ctx) // dist 拷贝（DIST_DIR）由 goroutine 收尾
```

```
checkProblematicFiles(ctx) // 源码树根出现 Android.mk/CleanSpec.mk 直接失败
checkRAM(ctx, config) // 内存低于阈值提前报错（dex2oat 是内存大户）
SetupOutDir(ctx, config) // 建 out/ 骨架目录（第四章 out/ 布局的来源）
SetupPath(ctx, config) // 用 prebuilts 组装 PATH
what := RunAll // 讲解注释④：阶段位掩码，默认五个阶段全开
if config.UseBazel() {
what = RunAllWithBazel // mixed build：把 RunNinja 换成 RunBazel
}
if config.SkipConfig() {
what = what &^ RunProductConfig // 讲解注释⑤：&^ 按位清零——调试时跳过阶段一
}
// SkipKati/SkipSoong/SkipNinja 同理各占一句清位
if what&RunProductConfig != 0 {
runMakeProductConfig(ctx, config) // 讲解注释⑥：阶段一=ckati dump 变量（第八章 §8.1）
}
// 讲解注释⑦：m installclean/dataclean 分支在此提前 return——只删安装产物/数据
if what&RunSoong != 0 {
runSoong(ctx, config) //
讲解注释⑧：阶段二=bootstrap+soong_build→out/soong/build.ninja（第七章）
}
if what&RunKati != 0 {
genKatiSuffix(ctx, config) // 讲解注释⑨：kati 文件后缀="-产品名"——切 lunch 即换名（§8.6）
runKatiCleanSpec(ctx, config) // 阶段三a：cleanbuild.mk 清旧安装产物
runKatiBuild(ctx, config) // 阶段三b：main.mk→out/build-<product>.ninja（第九章 §9.1）
runKatiPackage(ctx, config) // 阶段三c：packaging/main.mk→*-package.ninja
}
createCombinedBuildNinjaFile(ctx, config) // 讲解注释⑩：soong+kati 并线（§6.3）
// what&RunBuildTests 时在此跑 testForDanglingRules（悬空规则自检）
if what&RunNinja != 0 {
if what&RunKati != 0 {
installCleanIfNecessary(ctx, config) // 讲解注释⑪：切 lunch 自动 installclean（§6.5）
}
runNinjaForBuild(ctx, config) // 讲解注释⑫：阶段五——整个构建只有这一处真正执行编译
}
// what&RunBazel 分支收尾：mixed build 在此调 runBazel
}
```

对照 `m nothing` 为什么快：目标名 nothing 是 kati 生成的一个空 phony，五个阶段照常走完（尤其阶段一与 kati 的 `--regen` 判定，第九章 §9.2），但 ninja 没有可编的动作，秒级返回——"m nothing 验证语法"（第一章 1.2）验证的其实是阶段一至三。

6.3 combined.ninja：两个引擎并线

soong 与 kati 各自产出的 ninja 文件互不知晓，soong_ui 用一个 Go template 把它们"并线"成 combined 文件（实际路径 out/combined-<product>-<variant>.ninja，config.go:1279-1284）：

go build/soong/ui/build/build.go:61-70（原文+讲解注释）

```
var combinedBuildNinjaTemplate = template.Must(template.New("combined").Parse(`
builddir = {{.OutDir}} // 讲解注释①：声明构建目录 out/
{{if .UseRemoteBuild }}pool local_pool
depth = {{.Parallel}}
{{end -}}
pool highmem_pool // 讲解注释②：高内存池——dex2oat 等动作限并发（排障手册场景三）
```

```
depth = {{.HighmemParallel}}
{{if and (not .SkipKatiNinja) .HasKatiSuffix}}subninja {{.KatiBuildNinjaFile}} //
讲解注释③：out/build-
<product>.ninja
subninja {{.KatiPackageNinjaFile}} // 讲解注释④：out/build-<product>-package.ninja
{{end -}}
subninja {{.SoongNinjaFile}} // 讲解注释⑤：out/soong/build.ninja（soong_build 全部动作）
`))
```

subninja 与 include 的区别是这里的关键：subninja 的作用域隔离——kati 与 soong 两边可以各自定义同名 rule/variable 互不污染，只有 build 语句里的引用被打通；include 则是纯文本拼接、作用域共享。这正对应第五章"bp 与 mk 长期共存"的决策：两个世界的规则不必合并词法，只需在依赖边（soong 产出被 kati 侧 SOONG_ANDROID_MK 引用，第九章 §9.2）上衔接。

6.4 阶段位速查

阶段位	值	执行函数	产物
RunProductConfig	2	runMakeProductConfig	TARGET_DEVICE 等变量+构建横幅
RunSoong	4	runSoong	out/soong/build.ninja
RunKati	8	runKatiCleanSpec/Build/Package	out/build-<product>.ninja 与 *-package.ninja
RunKatiNinja	16	并入 combined 模板	combined 内的 subninja 行
RunNinja	32	runNinjaForBuild	全部编译/安装/镜像产物

三个特例目标不走位掩码：m help（build.go:156）、m clean（deleteBuildDir）、m installclean/dataclean（installClean/dataClean，删 out/target/product/<device>/ 下安装产物但保留 .intermediates——与第四章"空间回收"呼应）。

6.5 切 lunch 自动 installclean：previous_build_config.mk

阶段五开跑前有一次决定性检查（阶段机里唯一的"跨构建状态"）：

go build/soong/ui/build/cleanbuild.go:198-250（骨架+讲解注释）

```
func installCleanIfNecessary(ctx Context, config Config) {
configFile := config.DevicePreviousProductConfig() //
讲解注释①：out/target/product/<device>/previous
_build_config.mk
prefix := "PREVIOUS_BUILD_CONFIG := "
suffix := "\n"
currentConfig := prefix + config.TargetProduct() + "-" +
config.TargetBuildVariant() + suffix // 讲解注释②：当前配置=一行字符串"aosp_x86_64-userdebug"
ensureDirectoriesExist(ctx, filepath.Dir(configFile))
writeConfig := func() { ... } // 把 currentConfig 写回文件
previousConfigBytes, err := ioutil.ReadFile(configFile)
```

```
if err != nil {
if os.IsNotExist(err) {
writeConfig() // 讲解注释③：首次构建没有旧值——只记账，不清理
return
}
ctx.Fatalln("Failed to read previous product config:", err)
}
previousConfig := string(previousConfigBytes)
if previousConfig == currentConfig {
return // 讲解注释④：同一 lunch 目标——零成本返回，增量继续
}
if config.Environment().IsEnvTrue("DISABLE_AUTO_INSTALLCLEAN") {
// 讲解注释⑤：逃生开关——显式关闭自动清理（镜像可能与新配置不一致，后果自负）
ctx.Println("DISABLE_AUTO_INSTALLCLEAN is set and true; skipping auto-clean. " +
"Your tree may be in an inconsistent state.")
return
}
ctx.Printf("Build configuration changed: %q -> %q, forcing installclean\n",
previousProductAndVariant, currentProductAndVariant) // 讲解注释⑥：切换 lunch
后你见到的那行日志
installClean(ctx, config) // 讲解注释⑦：删镜像与安装目录，保留 .intermediates 中间产物
writeConfig() // 讲解注释⑧：记录新配置，下次构建以此为基线
}
```

为什么"只清安装产物、不清中间产物"安全：.intermediates 里的目标以输入哈希为键（第四章），换产品后键不冲突的部分继续复用；真正会混的是 system.img 等安装目录——它们是"当前产品"的函数，必须重装。这解释了切 lunch 的典型耗时（数分钟而非 2-4 小时）。

6.6 观测面：soong_ui 留在 out/ 的"黑匣子"

阶段机每一步都往 out/ 落盘观测数据——排障时先翻黑匣子再猜：

文件	写者与锚点	用途
out/soong.log	soong_ui 进程日志（cmd/soong_ui/main.go:196）	阶段机内部决策的详细流水
out/error.log	错误专用流（main.go:199，status/log.go:92）	排障第一入口（排障手册场景一）
out/.ninja_fifo	ninja 状态管道（ninja.go:39-44）	soong_ui 借它实时回显 ninja 进度
out/.ninja_log	ninja 执行账本（ninja.go:205）	查"谁被重编了"与卡死检测
out/soong/build_number.txt	main.mk:41-50	BUILD_NUMBER 落盘——命令行不抖动则不重编
out/soong/soong.environment.used.*	环境指纹（soong.go:66-67，config.go:918-920）	soong 重跑判定与环境对比（场景二）

两个容易忽略的机制值得点名。其一，**卡死检测**：ninja.go:204-234 的 ninjaStucknessChecker 以固定心跳比较 .ninja_log 的 mtime，若迟迟不动便打印 "ninja may be stuck" 并 `pstree -pal` 导出进程树——长构建"疑似挂死"时先看这段诊断而不是盲目 kill。其二，**build_number 落盘**（

main.mk:41-50 原注释 "Write the build number to a file...Avoids rebuilds when using ninja")：把易变的时间语义固化为文件内容比较，只有数字真变了才 mv——这是场景二"时间戳去抖"手法的官方范本。

> **设计洞察 · 逐段精读：观测面为什么分散在 out/ 各处**
> soong.log/error.log 在 out/ 根（soong_ui 的职责），环境指纹与 globs 在 out/soong/（soong_build 的职责），.ninja_log 在 out/ 根（ninja 的职责）——**谁写的数据落在谁的名字域**。知道"这个文件是谁写的"，等于知道"这个文件能回答哪一层的问题"：进程调度问 soong.log，图与动作问 .ninja_log，环境一致性问 environment.used。

> **设计洞察 · 设计洞察：阶段机的收敛之美**
> soong_ui 的五个阶段恰好是"信息逐步确定"的顺序：变量（阶段一）→ 模块图（阶段二）→ 依赖与规则（阶段三/四）→ 执行（阶段五）。任何阶段失败立即终止，不存在"边编译边发现产品配错"的浪费。位掩码设计让 m 的全部特例语义（nothing/installclean/skip 系列）都落在同一个函数里可读可测——这是把"命令行 UX"当作状态机来设计的范例。

第七章 Android.bp → build.ninja：soong_build 的两次遍历

> **设计洞察 · 学习目标**
> 1. 能按顺序说出 RunBlueprint 的五个阶段及每阶段的输入输出；2. 能解释 mutator 管道如何把一个模块定义展开成多架构/多变体实例；3. 能走读 ModuleBase.GenerateBuildActions 并指出模块作者真正要实现的钩子是哪一个；4. 能说明 restat 与 sbox --write-if-changed 如何支撑第四章的增量机制。

7.1 链路位置：RunSoong 阶段的内部

第六章阶段二 runSoong 调起 soong_build 二进制（日志原文："analyzing Android.bp files and generating ninja file at out/soong/build.ninja"，soong.go:260）。soong_build 的 main() 把所有工作委托给 blueprint 框架的 RunBlueprint（build/soong/cmd/soong_build/main.go:255-283：常规路径 stopBefore=bootstrap.DoEverything），后者是一条五步流水线：

步骤	入口函数	职责
A 找文件	ctx.ListModulePaths	按 moduleListFile 清单列出全部待解析 bp
B 解析	ctx.ParseFileList	bp 语法树求值，产出"模块定义"
C 变体与依赖	ctx.ResolveDependencies	跑全部 mutator：展开变体+解析依赖名

步骤	入口函数	职责
D 生成动作	ctx.PrepareBuildActions	逐模块调 GenerateBuildActions 产出 ninja 动作
E 写文件	ctx.WriteNinjaFile	写 out/soong/build.ninja 与 .d 依赖文件

第一步的 moduleListFile 由 soong_ui 的 finder 预扫描生成（out/.module_paths/Android.bp.list），所以 soong_build 不需要在启动时遍历整棵源码树——这是第四章"m services 比全编快"在解析层的伏笔。

7.2 RunBlueprint 走读

go build/blueprint/bootstrap/command.go:47-140（骨架+讲解注释）

```
func RunBlueprint(args Args, stopBefore StopBefore,
ctx *blueprint.Context, config interface{}) []string {
// 讲解注释①：soong_build 的 main() 最终只走到这里——五个阶段一次排开，
// stopBefore 用于"提前刹车"（--modulegraph 模式等）。
if args.ModuleListFile != "" {
ctx.SetModuleListFile(args.ModuleListFile)
} else {
fatalf("-l <moduleListFile> is required and must be nonempty")
}
ctx.BeginEvent("list_modules") // 讲解注释②：阶段A
filesToParse, err := ctx.ListModulePaths(srcDir)
if err != nil {
fatalf("could not enumerate files: %v\n", err.Error())
}
ctx.RegisterBottomUpMutator("bootstrap_plugin_deps", pluginDeps)
ctx.RegisterModuleType("bootstrap_go_package", newGoPackageModuleFactory())
ctx.RegisterSingletonType("bootstrap", newSingletonFactory())
// 讲解注释③：模块类型注册表——bp 里的每个类型名（如 cc_binary）映射到 Go 工厂
ctx.BeginEvent("parse_bp") // 讲解注释④：阶段B
blueprintFiles, errs := ctx.ParseFileList(".", filesToParse, config)
if len(errs) > 0 {
fatalErrors(errs)
}
// 讲解注释⑤：到这里"模块定义"只是属性包（properties），还没有变体、没有依赖
extraDeps, errs := ctx.ResolveDependencies(config) // 讲解注释⑥：阶段C
if len(errs) > 0 {
fatalErrors(errs)
}
// 讲解注释⑦：mutator 管道在跑：自顶向下的变体轴（arch→os→image…）
// 把每个模块定义克隆成 N 个变体实例，自底向上的 mutator 再解析依赖名
extraDeps, errs = ctx.PrepareBuildActions(config) // 讲解注释⑧：阶段D
if len(errs) > 0 {
fatalErrors(errs)
}
// 讲解注释⑨：内部三连——resolveDependencies 收尾、
// generateModuleBuildActions（逐模块）、generateSingletonBuildActions（逐单例）
if stopBefore == StopBeforeWriteNinja {
```

```
return ninjaDeps // 讲解注释⑩：只要模块图不要 ninja 的模式在此刹车
}
ctx.BeginEvent("write_files")
// 讲解注释⑪：阶段E——blueprint/ninja_writer.go 把全局规则/变量池+全部动作
// 写出 out/soong/build.ninja，同时写 .d 文件记录"哪些 bp/Go 文件影响本输出"
// ——ninja 据此决定下次构建要不要重跑 soong_build
}
```

PrepareBuildActions 的内部展开（build/blueprint/context.go:2390-2444）值得单独记忆：它先 resolveDependencies 兜底，再 generateModuleBuildActions、generateSingletonBuildActions，最后做"活跃变量引用环检查"（checkForVariableReferenceCycles，2437 行）——ninja 变量的 lazily-expansion 语义靠这一步保证。

7.3 变体展开：mutator 管道

soong/android 包在 init() 时注册全部模块类型与 mutator（build/soong/android/register.go:188-209）：

go build/soong/android/register.go:188-209（原文+讲解注释）

```
func (ctx *Context) Register() {
preSingletons.registerAll(ctx) // 讲解注释①：预备单例最先跑（读 config）
for _, t := range moduleTypes { // 讲解注释②：全部模块类型工厂注册——
t.register(ctx) // 每个包 init() 里 append 进 moduleTypes 的条目
}
mutators := collateGloballyRegisteredMutators()
mutators.registerAll(ctx) // 讲解注释③：变体轴 mutator 注册（arch/os/image/apex…）
// 第三章"变体爆炸"的机械成因就在这一步
singletons := collateGloballyRegisteredSingletons()
singletons.registerAll(ctx)
}
```

单例注册顺序里藏着刻意安排（register.go:196-209）：phony 排在 makevars 之前（phony 规则要以 Make 规则形式写进给 kati 的 make_vars 文件）；env/ninjadeps 永远最后（要统计所有模块用过的环境变量与 ninja 依赖，才能写 soong.environment.used 与 globs 文件——第六章 runSoong 的环境检查正靠它）。

7.4 GenerateBuildActions：框架钩子与模块钩子

阶段D 对每个模块调用 ModuleBase.GenerateBuildActions——这是"框架为你做的事"清单（build/soong/android/module.go:2186 起）：

go build/soong/android/module.go:2186-2262（骨架+讲解注释）

```
func (m *ModuleBase) GenerateBuildActions(blueprintCtx blueprint.ModuleContext) {
ctx := &moduleContext{
module: m.module,
bp: blueprintCtx, // 讲解注释①：包装 blueprint 原生 ctx——模块作者看到的是
android.ModuleContext
}
m.licenseMetadataFile = PathForModuleOut(ctx, "meta_lic") // 讲解注释②：许可元数据自动生成
dependencyInstallFiles, dependencyPackagingSpecs := m.computeInstallDeps(ctx)
```

```
// 讲解注释③：收集全部依赖的"安装产物"——cc_binary 安装时会自动带上其 shared 库，
// 第四章"adb sync 推增量"的文件清单就是从这个 DepSet 展开的
m.installFilesDepSet = newInstallFilesDepSet(nil, dependencyInstallFiles)
blueprintCtx.GetMissingDependencies()
// 讲解注释④：ALLOW_MISSING_DEPENDENCIES 模式（m mm 速编）的缺依赖上报点
if ctx.config.captureBuild { ctx.ruleParams = ... } // 调试用：记录规则参数
desc := "//" + ctx.ModuleDir() + ":" + ctx.ModuleName() + " "
// 讲解注释⑤：ninja 注释里的模块坐标——报错时看到的 //frameworks/base/libs/binder:libbinder
var suffix []string
if ctx.Os().Class != Device && ctx.Os().Class != Generic {
suffix = append(suffix, ctx.Os().String()) // 讲解注释⑥：host 模块加 [linux_glibc_x86_64] 后缀
}
if !ctx.PrimaryArch() {
suffix = append(suffix, ctx.Arch().ArchType.String()) // 非 64 位主架构再加 [arm]
}
if apexInfo := ctx.Provider(ApexInfoProvider).(ApexInfo); !apexInfo.IsForPlatform() {
suffix = append(suffix, apexInfo.ApexVariationName) // 讲解注释⑦：APEX 变体再加模块名后缀
}
checkDistProperties(ctx, "dist", &m.distProperties.Dist) // dist 属性校验
if m.Enabled() {
ctx.VisitDirectDepsBlueprint(func(bm blueprint.Module) {
if m, ok := bm.(Module); ok {
ctx.validateAndroidModule(bm, ...) // 讲解注释⑧：可见性校验（visibility 属性）在此执行
}
})
m.noticeFiles = make([]Path, 0)
// 讲解注释⑨：NOTICE/LICENSE 文件自动发现（notice 属性或同名文件）
m.module.GenerateAndroidBuildActions(ctx) // 讲解注释⑩：真正的模块钩子！
if ctx.Failed() {
return
}
// cc_binary 实现"怎么编译"，框架负责"编译前后的一切"
m.initRcPaths = PathsForModuleSrc(ctx, m.commonProperties.Init_rc)
rcDir := PathForModuleInstall(ctx, "etc", "init") // 讲解注释⑪：init_rc 自动装到
system/etc/init/
m.vintfFragmentsPaths = PathsForModuleSrc(ctx, m.commonProperties.Vintf_fragments)
// 讲解注释⑫：vintf_fragments 自动装到 system/etc/vintf/manifest/（书14 的 VINTF
三件套由此而来）
}
}
```

即：**模块作者只实现 GenerateAndroidBuildActions（module.go:1314 的文档示例），安装/许可/可见性/打包位置全部由基类钩子代劳**。第三章模块类型速查表里的每种类型，本质上就是"实现了 GenerateAndroidBuildActions 的一个 Go 结构体"。

7.5 RuleBuilder 与 restat：增量的机械基础

模块生成动作的现代方式是 RuleBuilder（build/soong/android/rule_builder.go:40-110）。两个关键能力：

go build/soong/android/rule_builder.go:40-110 与 634-638（骨架+讲解注释）

```
type RuleBuilder struct {
```

```
pctx PackageContext
ctx BuilderContext
commands []*RuleBuilderCommand
installs RuleBuilderInstalls
temporariesSet map[WritablePath]bool
restat bool // 讲解注释①：声明"本命令重跑后输出可能没变"
sbox bool // 讲解注释②：声明"本命令要在沙箱里原子写输出"
highmem bool // 讲解注释③：加入第六章的 highmem_pool
remoteable RemoteRuleSupports
rbeParams *remoteexec.REParams
outDir WritablePath
...
}
// Restat marks the rule as a restat rule, which will be passed to
// ModuleContext.Rule in BuildParams.Restat.
func (r *RuleBuilder) Restat() *RuleBuilder { // 讲解注释④：链式 API
r.restat = true
return r
}
...
if r.restat {
sboxCmd.Flag("--write-if-changed") // 讲解注释⑤：sbox 沙箱比较新旧输出，内容没变就不动文件
}
```

restat 的语义：ninja 重跑某条规则后，若输出文件内容与跑之前相同（restat 会重置其 mtime 为旧值），ninja 判定"下游无需重编"——大量代码生成器（aidl、protobuf、资源链接）因此不会因时间戳抖动引发重编风暴。sbox 则把"多条命令写多个输出"包进沙箱：输出先写临时目录再整体提交，配合 --write-if-changed 保证"输出不变则时间戳不变"。这两件武器是第四章"增量只重编哈希变化的模块"在单条动作粒度上的补全。

7.6 全图的落盘形态：build.ninja 之外还有什么

阶段E 不是只写一个文件。soong_build 交付物全家福（均可在 out/soong/ 找到实据）：

- 1. build.ninja——第七章全部动作的主体，第六章以 subninja 并线；
- 2. build.ninja.d——writeDepFile（cmd/soong_ui/main.go:286-295）写出的 make 风格依赖：列出影响本输出的全部 bp/Go 文件，ninja 据此决定 soong_build 是否需要重跑；
- 3. globs-*.ninja——writeBuildGlobsNinjaFile（main.go:284-285）为每处 `glob` 生成依赖规则，目录内容变化会触发 soong_build 重跑（glob 是 bp 少数"动态"来源）；
- 4. soong.environment.used.<tag>——用到的环境变量清单（soong.go:66-67 定义常量），环境一致性比对基准；
- 5. make_vars-<product>.mk / late-<product>.mk / installs-<product>.mk——第七章 makeVarsSingleton 写给 Kati 的合同（第九章 §9.2）。

模块级依赖图另有专门产物：`m module-info` 产出 out/target/product/<device>/module-info.json（main.mk:1676-1681 经 dist-for-goals 分发，受 SOONG_COLLECT_JAVA_DEPS 门控；Java 模块依赖另落 module_bp_java_deps.json）。第一章 1.2 的"m module-info 查依赖"命令输出的就是这份全图快照——排障时用它验证"我以为的依赖边"与"soong 实际展开的边"是否一致，是分析题

12 的实操入口。

> **设计洞察 · 设计洞察：两遍模型的本质**
> bp 是声明式的：模块 A 写 shared_libs: ["libbinder"] 时，libbinder 可能还没解析，甚至还不存在（后解析的文件才定义它）。所以必须第一遍（阶段C）先把全图建完备、变体全部展开、名字全部解析，第二遍（阶段D）才允许模块查询依赖、计算输出路径。这与 GC 的 mark-sweep、编译器的前端-后端同构：**把"未知"收敛成"已知"再动手**，是所有静态分析系统的通用纪律。

第八章 product 配置继承图：inherit-product 与 rbc 双轨

> **设计洞察 · 学习目标**
> 1. 能说清 AndroidProducts.mk 如何声明 lunch 目标以及 _read-ap-file 的三重校验；2. 能解释 inherit-product 的"@inherit: 占位符记账"机制与 import-nodes 的后序 DAG 展开；3. 能描述 Android 13 中 rbc/Starlark 产品配置的灰度开关与数据流；4. 能从源码角度回答"切 lunch 为什么重跑 kati 且自动 installclean"。

8.1 链路位置：第六章阶段一的内部

第二章讲过 device 树三层继承；本章把它下钻到变量级。第六章阶段一 runMakeProductConfig 并不直接跑 main.mk——它调 dumpMakeVars（build/soong/ui/build/dumpvars.go:83-110），本质是再开一个 ckati 进程：`ckati -f build/make/core/config.mk dump-many-vars`，把 TARGET_DEVICE/TARGET_DEVICE_DIR 等几十个变量 dump 出来填进 soong_ui 的环境与 config 对象（dumpvars.go:268-283）。config.mk 一路 include 进 product_config.mk——**产品配置引擎**。所以"lunch 的实质"是：先跑一次轻量 kati 只为取变量。

> **设计洞察 · 逐段精读：为什么阶段一要用另一个 ckati 进程**
> 产品配置（PRODUCT_*）与构建规则（模块怎么编）是两个知识域。用独立进程 dump 变量，可以把产品配置的错误（拼错产品名、缺 PRODUCT_NAME）在阶段一就地引爆，而不是拖到解析数万行 main.mk 的阶段三才炸。隔离也带来缓存友好：dump 结果可对拍、可测试（build/make/tests/ 下成套的 rbc 测试正是这么做的）。

8.2 AndroidProducts.mk：产品注册表

lunch 目标从哪来？每个 device 目录的 AndroidProducts.mk。它由 finder 预扫描成清单，product_config.mk:152 读入：`android_products_makefiles := $(file <$(OUT_DIR)/.module_paths/AndroidProducts.mk.list)`。以 device/google/cuttlefish/AndroidProducts.mk:18-24 为例，条目是"产品名:路径"对：

aosp_cf_x86_64_phone:$(LOCAL_DIR)/vsoc_x86_64/phone/aosp_cf.mk。解析函数是 _read-ap-file：

mk build/make/core/product_config.mk:178-208（原文+讲解注释）

```
# Reads given AndroidProduct.mk file and sets the following variables:
define _read-ap-file
$(eval PRODUCT_MAKEFILES :=) \
$(eval COMMON_LUNCH_CHOICES :=) \
$(eval STARLARK_OPT_IN_PRODUCTS := ) \
# 讲解注释①：每次读一个 AndroidProducts.mk 前先清空三个变量——防止文件间串值
$(eval LOCAL_DIR := $(patsubst %/,%,$(dir $(f)))) \
$(eval include $(f)) \
# 讲解注释②：include 执行设备自己的 AndroidProducts.mk，填入上面三个变量
$(foreach p, $(PRODUCT_MAKEFILES),$(eval ap_product_paths += $(call _product-spec,$(p)))) \
# 讲解注释③：_product-spec 把条目规范成 <product>:<path> 对——
# 写成 path/to/<product>.mk 的老格式自动补产品名
$(eval ap_common_lunch_choices := $(COMMON_LUNCH_CHOICES)) \
$(eval ap_products_using_starlark_config := $(STARLARK_OPT_IN_PRODUCTS)) \
$(eval _products := $(call _first,$(ap_product_paths),:)) \
$(eval _bad := $(filter-out $(_products),$(call _first,$(ap_common_lunch_choices),-))) \
$(if $(_bad),$(error COMMON_LUNCH_CHOICES contains products(s) not defined in this file:
$(_bad)))
\
# 讲解注释④：校验一——lunch 菜单里出现的产品必须在本文件 PRODUCT_MAKEFILES 中声明
$(eval _bad := $(filter-out %-eng %-userdebug %-user,$(ap_common_lunch_choices))) \
$(if $(_bad),$(error invalid variant in COMMON_LUNCH_CHOICES: $(_bad)))
# 讲解注释⑤：校验二——变体只能是 eng/userdebug/user 三选一
$(eval _bad := $(filter-out $(_products),$(ap_products_using_starlark_config))) \
$(if $(_bad),$(error STARLARK_OPT_IN_PRODUCTS contains product(s) not defined in this file:
$(_bad
)))
# 讲解注释⑥：校验三——rbc 灰度名单里的产品必须真实存在（§8.4）
endef
```

聚合之后（product_config.mk:210-228），soong_ui 阶段一还负责重名检测：同一产品名出现在两个文件直接 error（"Product name must be unique"）。这就是为什么 lunch 列表永远不会出现歧义项。

8.3 inherit-product：不是 include，是记账

第二章说"每层 device.mk 叠加 PRODUCT_PACKAGES"——机械上靠的不是 shell 变量叠加，而是一套延迟展开协议。define inherit-product（build/make/core/product.mk:395-409）：

mk build/make/core/product.mk:395-409（原文+讲解注释）

```
define inherit-product
$(eval _inherit_product_wildcard := $(wildcard $(1)))\
$(if $(_inherit_product_wildcard),,$(error $(1) does not exist.))\
# 讲解注释①：继承目标不存在立即 error——不等到运行期才炸
$(foreach part,$(_inherit_product_wildcard),\
$(if $(findstring ../,$(part)),\
$(eval np := $(call normalize-paths,$(part))),\
$(eval np := $(strip $(part))))\
$(foreach v,$(_product_var_list), \
```

```
$(eval $(v) := $($(v)) $(INHERIT_TAG)$(np))) \
# 讲解注释②：核心一行！对每个产品变量（PRODUCT_PACKAGES/PRODUCT_COPY_FILES/…）
# 追加一个占位符 "@inherit:路径"——父产品的值此刻并没进来，只是记了"账"
$(eval current_mk := $(strip $(word 1,$(_include_stack)))) \
$(eval inherit_var := PRODUCTS.$(current_mk).INHERITS_FROM) \
$(eval $(inherit_var) := $(sort $($(inherit_var)) $(np))) \
# 讲解注释③：同时记录本产品文件的继承图边——INHERITS_FROM 是 DAG 的邻接表
$(call dump-inherit,$(strip $(word 1,$(_include_stack))),$(1)) \
$(call dump-config-vals,$(current_mk),inherit))
endef
```

INHERIT_TAG 定义在 build/make/core/node_fns.mk:109：`INHERIT_TAG := @inherit:`。所以产品 makefile 里写 `$(call inherit-product, device/google/cuttlefish/vsoc_x86_64/phone/aosp_cf.mk)` 之后，本文件的每个 PRODUCT_* 变量值里都埋了一串 `@inherit:...` 占位符——继承关系变成了**数据**而非 include 的执行副作用。

8.4 import-nodes：DAG 后序展开

占位符由 import-nodes 体系展开（product.mk:450 的 import-products 即它的薄封装）。每个节点（产品 makefile）的展开函数 _import-node（node_fns.mk:193-216）：

mk build/make/core/node_fns.mk:193-216（原文+讲解注释）

```
define _import-node
$(eval _include_stack := $(2) $$(_include_stack))
$(call clear-var-list, $(3))
# 讲解注释①：清空全部产品变量——本节点的工作区
$(eval LOCAL_PATH := $(patsubst %/,%,$(dir $(2))))
$(eval MAKEFILE_LIST :=)
$(eval include $(2))
# 讲解注释②：真正 include 产品 mk——它内部的 inherit-product 只埋占位符，
# 不会触发父文件的再次 include（对比普通 include 的递归执行）
$(eval _included := $(filter-out $(2),$(MAKEFILE_LIST)))
$(eval MAKEFILE_LIST :=)
$(call copy-var-list, $(1).$(2), $(3))
$(call clear-var-list, $(3))
# 讲解注释③：把本节点的"占位符版本"存到命名空间 PRODUCTS.<mk>.<var> 下
$(eval $(1).$(2).inherited := \
$(call get-inherited-nodes,$(1).$(2),$(3)))
# 讲解注释④：从占位符反查出父节点列表
$(call _import-nodes-inner,$(1),$($(1).$(2).inherited),$(3),$(4))
# 讲解注释⑤：递归展开父节点——.seen 守卫保证钻石继承只展开一次
$(call _expand-inherited-values,$(1),$(2),$(3),$(4))
# 讲解注释⑥：把 "@inherit:父" 占位符替换成父节点展开后的真值（node_fns.mk:148-191）
$(eval $(1).$(2).inherited :=)
$(eval _include_stack := $(wordlist 2,9999,$$(_include_stack)))
endef
```

_expand-inherited-values 的裁决规则值得背下来：列表变量（PRODUCT_PACKAGES 等）把父值**原位拼进占位符位置**——即子产品写在 inherit-product 之后的条目排在父值之前，控制拼接顺序即可控制优先级；单值变量（PRODUCT_NAME 等）若子节点已赋值则**丢弃父值**（"子赢"）；同一占位符出现两次用 uniq-word 去重，父值被清空防止 DAG 重复展开；若展开后发现还剩指向自己的

@inherit: 标签，则报 "Cycle detected"（node_fns.mk:184-188）。

> **设计洞察 · 逐段精读：为什么用占位符而不用直接 include**
> 直接 include 父 makefile 是"执行语义"——变量当场求值，继承顺序与 include 顺序纠缠，钻石继承必重复。占位符把继承变成可分析的数据结构：DAG 可以先建好（INHERITS_FROM）、后展开（_expand-inherited-values），等价于把"多重继承线性化"这个难题从执行期挪到了图算法期。rbc 迁移也受益于此——继承图能被 Starlark 一比一重放。

8.5 rbc 双轨：Android 13 的现状

Android 13 的产品配置主路径仍是上述 mk 引擎，但 Starlark 迁移（项目代号 rbc）已经铺好双轨。灰度开关在 product_config.mk:231-251：

mk build/make/core/product_config.mk:231-251（骨架+讲解注释）

```
ifneq (,$(filter $(TARGET_PRODUCT),$(products_using_starlark_config)))
RBC_PRODUCT_CONFIG := true
RBC_BOARD_CONFIG := true
endif
# 讲解注释①：只对 AndroidProducts.mk 里 STARLARK_OPT_IN_PRODUCTS 点名的产品启用 rbc
ifndef RBC_PRODUCT_CONFIG
$(call import-products, $(current_product_makefile))
# 讲解注释②：传统路径——§8.3/§8.4 的 mk 引擎
else
$(shell mkdir -p $(OUT_DIR)/rbc)
$(call dump-variables-rbc, $(OUT_DIR)/rbc/make_vars_pre_product_config.mk)
# 讲解注释③：先把 mk 侧"输入变量"落盘，作为 rbc 脚本的输入契约
$(shell build/soong/scripts/update_out \
$(OUT_DIR)/rbc/rbc_product_config_results.mk \
build/soong/scripts/rbc-run \
$(current_product_makefile) \
$(OUT_DIR)/rbc/make_vars_pre_product_config.mk)
# 讲解注释④：rbc-run 解释执行产品对应的 .rbc 文件，产出与传统路径同格式的结果
ifneq ($(.SHELLSTATUS),0)
$(error product configuration converter failed: $(.SHELLSTATUS))
endif
include $(OUT_DIR)/rbc/rbc_product_config_results.mk
# 讲解注释⑤：结果以 mk 形式回灌——下游（main.mk 等）完全无感知
endif
```

即：rbc 时代产品配置"计算"用 Starlark，"交付"仍是 mk 变量文件——下游 Kati/Soong 无缝衔接。build/make/tests/ 下有 product.rbc、board.rbc 等对拍用例，保证双轨结果一致。本树 build/make/core/product_config.rbc 已存在，是迁移的样板。

8.6 切 lunch 的完整因果链

把第六章与本章的知识合起来，"切 lunch 为什么要重跑 kati"有了全链答案：

- 1. genKatiSuffix（build/soong/ui/build/kati.go:43-48）把产品名编进文件名：katiSuffix := "-" + config.TargetProduct()——combined-<product>-<variant>.ninja、

build-<product>.ninja 全套换名，对 ninja 而言是全新输出；

- 2. kati 本身有 --regen 增量判定（kati.go:93-95 注释原文："Regenerate the Ninja file if environment inputs have changed. e.g. CLI flags, .mk file timestamps, env vars, $(wildcard ..) and some $(shell ..) results"）——TARGET_PRODUCT 变了属于 env/CLI 变化，必须重算；
- 3. installCleanIfNecessary（cleanbuild.go:198）比对 previous_build_config.mk 的 product-variant 字符串，变了就 installclean（§6.5）。

8.7 调试继承图工具箱

继承图出问题（某个 PRODUCT_PACKAGES 不生效、某变量"莫名"被覆盖）时，沿着记账机制逐层取证：

- 1. 看记账边：inherit-product 把每条继承写进 PRODUCTS.<mk>.INHERITS_FROM 命名空间（product.mk:406-408），并同步调 dump-inherit/dump-config-vals 钩子——这两个钩子是配置 dump 的取样点，可在 dump 输出里直接看到继承链；
- 2. 手工问变量：`get_build_var TARGET_DEVICE_DIR`（envsetup.sh:138）实为 `soong_ui.bash --dumpvar-mode TARGET_DEVICE_DIR`（cmd/soong_ui/main.go:75-80），一次轻量 ckati dump，不触发构建；
- 3. 批量对拍：--dumpvars-mode（main.go:83-88）以 shell 语法一次吐多个变量，适合脚本对比两个产品的配置差异；
- 4. 大规模回归：build/soong/cmd/multiproduct_kati 对数百个产品配置各跑一次 kati 产 ninja（kati.go:131-137 的 --empty_ninja_file 注释明确说它"runs Kati several hundred times for different configurations"且跳过写文件以防数百 GB 磁盘写），是产品配置的 CI 压测工具；
- 5. rbc 对拍：§8.5 的 dump-variables-rbc（product_config.mk:240）把 mk 侧输入变量落盘到 out/rbc/，是双轨一致性的比较基准。

> **设计洞察 · 逐段精读：为什么调试配置要先看"账本"再看"值"**
> 直接看最终值（如 PRODUCT_PACKAGES 列表）无法区分"没人加过"与"被覆盖"；而 INHERITS_FROM/占位符账本记录的是过程。这与书3 调 Binder 时"先抓调用栈再看返回值"同一方法论：**机制类问题要取证过程，结果类问题才看终态**。

> **设计洞察 · 设计洞察：配置即依赖**
> 这三层机制（换文件名、env 指纹、状态文件比对）本质是同一件事：把"我上次是用什么配置编的"做成可比较的依赖边。任何构建系统（bazel 的 --stamp、cargo 的 fingerprint）都要回答这个问题；Android 的答案朴素但分层清楚——文件名管粗粒度，--regen 管细粒度，previous_build_config 管安装产物一致性。调试"切配置没生效"类问题，就沿这三层逐层查（排障手册场景二）。

第九章 Kati 边界与打包签名：从 .mk 到 OTA 包

设计洞察 · 学习目标

1. 能读懂 runKati 传给 ckati 的完整参数表并解释 --regen/-ninja/no_ninja_prelude 三个关键开关；2. 能指出 Soong 模块"搬进" Make 世界的三个注入文件（installs/Android-<product>/late-<product>.mk）；3. 能走读 otapackage 规则与 GenerateAbOtaPackage 的 A/B 包生成流程；4. 能说出 target-files、super_empty.img、ReplaceCerts 在签名链路中的角色。

9.1 链路位置：镜像段的两条腿

第一章管线的第④段"镜像打包"由两条腿支撑：Kati 把 build/make/core/main.mk 翻译成 ninja 规则（本机执行），releasetools 把编译产物组装成发行物（OTA/签名）。本册至此已走完 soong_ui（第六章）、soong_build（第七章）、product（第八章），本章补上最后两环。

Kati 不是 make 的插件，而是 soong_ui 直接调用的独立二进制 ckati（prebuilts/build-tools/<host>/bin/ckati，config.go:1333 的 PrebuiltBuildTool）。它在阶段三被调用三次：runKatiCleanSpec（清增量）、runKatiBuild（主翻译）、runKatiPackage（打包规则），三者共用同一个 runKati 基函数：

go build/soong/ui/build/kati.go:73-141（骨架+讲解注释）

```
func runKati(ctx Context, config Config, extraSuffix string,
args []string, envFunc func(*Environment)) {
executable := config.PrebuiltBuildTool("ckati")
args = append([]string{
// Mark the output directory as writable.
"--writable", config.OutDir() + "/",
// Fail when encountering implicit rules.
"--werror_implicit_rules", // 讲解注释①：%.foo 隐式规则直接报错——禁用 make 老古董
// Entry point for the Kati Ninja file generation.
"-f", "build/make/core/main.mk", // 讲解注释②：Make 世界的总入口（2000+ 行）
...
"SOONG_MAKEVARS_MK=" + config.SoongMakeVarsMk(),
// 讲解注释③：out/soong/make_vars-<product>.mk——soong 导出的"变量桥"
"SOONG_ANDROID_MK=" + config.SoongAndroidMk(),
// 讲解注释④：out/soong/Android-<product>.mk——soong 模块翻成 kati 模块的总账本
"TARGET_DEVICE_DIR=" + config.TargetDeviceDir(),
"KATI_PACKAGE_MK_DIR=" + config.KatiPackageMkDir(),
}, args...)
...
// Instead of executing commands directly, generate a Ninja file.
"--ninja", // 讲解注释⑤：核心模式——kati 只翻译不执行
// Generate Ninja files in the output directory.
"--ninja_dir=" + config.OutDir(),
// Filename suffix of the generated Ninja file.
"--ninja_suffix=" + config.KatiSuffix() + extraSuffix,
// 讲解注释⑥：产出 out/build-<product>.ninja（后缀即第八章 §8.6 的文件名机制）
// Remove common parts at the beginning of a Ninja file...
```

```
"--no_ninja_prelude", // 讲解注释⑦：省去公共头部，方便 combined 并线
// Regenerate the Ninja file if environment inputs have changed. e.g.
// CLI flags, .mk file timestamps, env vars, $(wildcard ..) and some
// $(shell ..) results.
"--regen", // 讲解注释⑧：增量判定开关——输入没变就不重生成 ninja
...
// Detect the use of $(shell echo ...).
"--detect_android_echo", // 讲解注释⑨：捕获 shell 输出作为依赖指纹的一部分
// Use the built-in emulator of GNU find for better file finding
// performance. Used with $(shell find ...).
"--use_find_emulator", // 讲解注释⑩：find 仿真——避免真 fork 数千个 find 进程
...
"--werror_real_to_phony", // 讲解注释⑪：真目标依赖 phony 直接失败——严格性从这里来
"--werror_phony_looks_real",
...
"--kati_stats", // 讲解注释⑫：结束时打印变量数/规则数统计——排障利器
}
```

runKatiBuild（kati.go:190-226）在此之上补 `--werror_overriding_commands`（重复定义目标报错，除非 BUILD_BROKEN_DUP_RULES），产出 out/build-<product>.ninja；runKatiPackage（kati.go:302-315）则换入口 `-f build/make/packaging/main.mk` 产出 *-package.ninja——dist-for-goals 等"发行物分发"规则单独一册账本。三者都在第九章阶段三内完成，产物由第六章 combined.ninja 的 subninja 并线。

9.2 main.mk 的边界：Soong 模块怎么"搬进" Make 世界

main.mk 开头（build/make/core/main.mk:1-5）就立了规矩：

mk build/make/core/main.mk:1-5 与 537-541（原文+讲解注释）

```
ifndef KATI
$(warning Calling make directly is no longer supported.)
$(warning Either use 'envsetup.sh; m' or 'build/soong/soong_ui.bash --make-mode')
$(error done)
endif
# 讲解注释①：裸跑 make 立即失败——main.mk 依赖 kati 的扩展（.KATI_READONLY 等）
...
subdir_makefiles := $(SOONG_OUT_DIR)/installs-$(TARGET_PRODUCT).mk $(SOONG_ANDROID_MK)
# Android.mk files are only used on Linux builds, Mac only supports Android.bp
ifeq ($(HOST_OS),linux)
subdir_makefiles += $(file <$(OUT_DIR)/.module_paths/Android.mk.list)
endif
subdir_makefiles += $(SOONG_OUT_DIR)/late-$(TARGET_PRODUCT).mk
$(foreach mk,$(subdir_makefiles),$(info [$(call inc_and_print,...)/...] including $(mk)
...)$(eval i
nclude $(mk)))
# 讲解注释②：Kati 侧"看到"的模块来源只有四类：
# installs-<product>.mk——soong 模块的安装路径清单（makevars.go:229-250 生成）
# Android-<product>.mk——soong 模块翻译成 kati 模块的总账本（第七章动作的 mk 投影）
# Android.mk.list——全树存量 Android.mk（linux 专用）
# late-<product>.mk——单例们迟到的变量（dist 目标等）
```

生成这批文件的正是 soong 侧的 makeVarsSingleton（build/soong/android/makevars.go:229-250）：它写 make_vars-<product>.mk、late-<product>.mk、installs-<product>.mk 三个文件——Kati 与 Soong 的全部接口就浓缩在这三个文件加一个环境变量约定里。第六章单例注册顺序（register.go:196-209）"phony 在 makevars 前"的原因也在此：phony 规则要以 Make 语法写进这批文件。

> **设计洞察 · 设计洞察：边界即合同**
> soong 与 kati 各自内部可以大改（blueprint 重写、kati 加缓存），但三文件一变量（make_vars/installs/late + SOONG_ANDROID_MK）是十年未变的合同。类比书3 的 Binder：进程边界上的协议稳定，边界内各自演进——大型系统的可维护性来自"把不稳定面压到最小"。

9.3 target-files：镜像的"中间发行物"

Kati 侧 main.mk 最终声明出 droid/snod/otapackage 等目标；其中 BUILT_TARGET_FILES_PACKAGE 是关键中间产物——一个 zip，按 IMAGES/、SYSTEM/、VENDOR/、META/ 结构收纳所有已编译模块与分区镜像。组装者是 releasetools 的 add_img_to_target_files.py：AddImagesToTargetFiles（758 行）逐分区调 build_image.py 生成 ext4/erofs 镜像，动态分区设备再生成 super_empty.img（664 行 CreateSuperEmptyImage；987-988 行按 BOARD_BUILD_SUPER_EMPTY 分支打 banner）。

super_empty.img 是动态分区时代的"分区布局说明书"：只含 super 分区的元数据（各组 group、各逻辑分区 system/vendor/product 的尺寸与属组），不含数据——fastbootd 先刷它建好布局，再逐个刷逻辑分区。旧式 recovery 线刷时代"改分区表要重打整张 super.img"的痛点由此消除。

9.4 otapackage：mk 侧的打包规则

OTA 包的生成入口是 build/make/core/Makefile:6107-6150 的规则（这是 Kati 翻译成 ninja 的一条真实动作）：

mk build/make/core/Makefile:6107-6150（骨架+讲解注释）

```
ifeq ($(build_ota_package),true)
# -----------------------------------------------------------------
# OTA update package
# $(1): output file
# $(2): additional args
define build-ota-package-target
PATH=$(INTERNAL_USERIMAGES_BINARY_PATHS):$(dir $(ZIP2ZIP)):$$PATH \
$(OTA_FROM_TARGET_FILES) \
--verbose \
--extracted_input_target_files $(patsubst %.zip,%,$(BUILT_TARGET_FILES_PACKAGE)) \
--path $(HOST_OUT) \
$(if $(OEM_OTA_CONFIG), --oem_settings $(OEM_OTA_CONFIG)) \
$(2) \
$(BUILT_TARGET_FILES_PACKAGE) $(1)
endef
# 讲解注释①：打包=调 ota_from_target_files.py 吃 target-files 吐 zip——
```

```
# Python 工具链承担"发行物组装"，mk 只负责排依赖与传参
product_name := $(TARGET_PRODUCT)
ifeq ($(TARGET_BUILD_TYPE),debug)
product_name := $(product_name)_debug
endif
name := $(product_name)-ota-$(FILE_NAME_TAG)
INTERNAL_OTA_PACKAGE_TARGET := $(PRODUCT_OUT)/$(name).zip
INTERNAL_OTA_METADATA := $(PRODUCT_OUT)/ota_metadata
$(INTERNAL_OTA_PACKAGE_TARGET): KEY_CERT_PAIR := $(DEFAULT_KEY_CERT_PAIR)
$(INTERNAL_OTA_PACKAGE_TARGET): .KATI_IMPLICIT_OUTPUTS := $(INTERNAL_OTA_METADATA)
# 讲解注释②：kati 扩展——一条规则两个输出（包体+元数据）
$(INTERNAL_OTA_PACKAGE_TARGET): $(BUILT_TARGET_FILES_PACKAGE) $(OTA_FROM_TARGET_FILES)
$(INTERNAL_OT
ATOOLS_FILES)
@echo "Package OTA: $@"
$(call build-ota-package-target,$@,-k $(KEY_CERT_PAIR) --output_metadata_path
$(INTERNAL_OTA_MET
ADATA))
# 讲解注释③：依赖三件套——target-files 本体、脚本自身、otatools 全家桶
# （改 ota_from_target_files.py 任何一行的重打包都由这条依赖边驱动）
.PHONY: otapackage
otapackage: $(INTERNAL_OTA_PACKAGE_TARGET)
endif
```

9.5 A/B 包生成与签名

GenerateAbOtaPackage（build/make/tools/releasetools/ota_from_target_files.py:1123-1230）是 A/B 设备 OTA 的核心流程：

text build/make/tools/releasetools/ota_from_target_files.py:1123-1230（走读+讲解注释）

```
def GenerateAbOtaPackage(target_file, output_file, source_file=None):
# 讲解注释①：source_file 非空即增量 OTA（-i 传入旧版本 target-files）
output_zip = zipfile.ZipFile(staging_file, "w", ...)
if source_file is not None:
# 讲解注释②：增量包需要新旧两份 META/ab_partitions.txt 对齐
target_info = common.BuildInfo(OPTIONS.target_info_dict, OPTIONS.oem_dicts)
source_info = common.BuildInfo(OPTIONS.source_info_dict, OPTIONS.oem_dicts)
# VABC=Virtual A/B Compression（虚拟 A/B 压缩快照）
if not source_info.is_vabc or not target_info.is_vabc:
logger.info("Either source or target does not support VABC, disabling.")
OPTIONS.disable_vabc = True
# 讲解注释③：VABC 要求源端 update_engine 能写压缩操作、
# 目标端 snapuserd 能解释——任一端缺能力就退化为非压缩，防变砖
if OPTIONS.retrofit_dynamic_partitions:
target_file = GetTargetFilesZipForRetrofitDynamicPartitions(...)
# 讲解注释④：旧设备改造动态分区的 OTA 要重排 target-files 里的分区清单
...
CheckVintfIfTrebleEnabled(target_file, target_info)
# 讲解注释⑤：升级前后 VINTF 兼容性预检（书14 的 VINTF 矩阵在此把关）
metadata = GetPackageMetadata(target_info, source_info)
# 讲解注释⑥：OTA 元数据（pre/post 条件：指纹、时间戳、分区状态）
payload = Payload()
...
```

```
max_timestamp = str(metadata.postcondition.timestamp)
partition_timestamps_flags = GeneratePartitionTimestampFlags(...)
# 讲解注释⑦：防回滚——payload 记录各分区最大可接受时间戳
...
additional_args += ["--enable_zucchini", str(OPTIONS.enable_zucchini).lower()]
# 讲解注释⑧：zucchini/lz4diff 是增量差分算法开关，按新旧版本能力自动启停
```

签名侧的对应物是 sign_target_files_apks.py：它吃未签名 target-files，按 -e <apk>=<key> 映射逐 APK 重签，并执行 ReplaceCerts（783-800 行——把 SELinux MAC 策略文件等文本里的旧证书指纹整体替换为新证书，防 frameworks/base/services/core/java/com/android/server/pm/SELinuxMMAC.java 的重复证书断言失败）与 ReplaceOtaKeys（920 行——重写 META/otakeys.txt）。产出签名版 target-files 后再跑一次 ota_from_target_files，才是可发行的 OTA 包。

9.6 发行物速查：releasetools 一览

build/make/tools/releasetools/ 的主要工具与本书章节的对应：

目标/工具	输入→输出	锚点
add_img_to_target_files.py	staging 产物→完整 target-files zip	AddImagesToTargetFiles（758 行）
build_super_image.py	misc_info→super.img/super_empty.img	BuildSuperImage（181 行）
ota_from_target_files.py	target-files→OTA 包	GenerateAbOtaPackage（1123 行）
img_from_target_files.py	target-files→整包镜像 zip	与 ota 同吃一份 target-files
sign_target_files_apks.py	未签名 target-files→签名版	ReplaceCerts（783 行）
check_target_files_signatures.py	签名版 target-files→校验报告	签名一致性的 CI 关卡

同一份 target-files 可派生 OTA 包、整包镜像、增量差分基线——这正是下方 box 所述"中间发行物"设计的收益面。日常自检的最小闭环：`m otapackage` 后用 check_target_files_signatures.py 复核，再 adb 侧验证升级。

设计洞察 · 逐段精读：为什么打包要绕道 target-files

target-files 是"一份构建、多处发行"的中间表示：同一份可以出 OTA 包、img_from_target_files 出整包镜像、merge 目标合 system/vendor 两半构建、VABC 差分工具做增量。若直接从 staging 目录打包，签名重排/分区重配就要反复动原始产物；有了中间发行物，"编译"与"发行"彻底解耦——与书6 的 APK 安装签名流程同一设计哲学。

第十章 APEX 构建：从 apex{} 到 Mainline 交付物

设计洞察 · 学习目标

1. 能走读 apexBundle.GenerateAndroidBuildActions 的四步结构并说出每步产物；2. 能解释 apexRule 一条 ninja 规则如何封装 mke2fs/SELinux 标签/AVB 签名的完整工具链；3. 能说明 linkToSystemLib 符号链接优化的条件与为何可更新 APEX 被排除；4. 能给出压缩 APEX（.capex）的启用条件链。

10.1 链路位置：第七章变体轴的"第四维"落地

第三章的模块类型速查表里 apex{} 只有一行；第七章讲变体轴时 APEX 是最后一条轴（module.go 的 suffix 逻辑，§7.4）。本章把这最后一环展开：一个 apex{} 模块从被解析到产出 out/soong/.intermediates/<path>/<name>/android_common/<name>.apex（随后由 Kati 侧安装到 out/target/product/<device>/system/apex/，衔接第九章 target-files 的 SYSTEM/apex 目录），中间经历什么。它是"第七章两遍模型"的最复杂消费者：APEX 自己不产生代码，它的工作是把**其他模块的产物**按清单重新组织、再打成带签名的载荷镜像——所以 GenerateBuildActions 的主体是"遍历依赖树收集文件"而不是"编译"。

10.2 GenerateAndroidBuildActions 四步走读

apex.go:1778 的函数体自带四步大注释（1 校验/2 收集/3 配置/4 生成规则），直接沿注释走读：

go build/soong/apex/apex.go:1778-1815 与 2136-2175（骨架+讲解注释）

```
func (a *apexBundle) GenerateAndroidBuildActions(ctx android.ModuleContext) {
// 1) do some validity checks such as apex_available, min_sdk_version, etc.
a.checkApexAvailability(ctx) // 讲解注释①：apex_available 反向校验——依赖库必须声明
a.checkUpdatable(ctx) // "允许被本 APEX 使用"，违者在解析期就报错（ch7 可见性思想的延伸）
... （其余四项校验：CheckMinSdkVersion/checkStaticLinkingToStubLibraries/
// checkStaticExecutables/apex_test 专属属性检查，同处 1783-1792 行）
// 2) traverse the dependency tree to collect apexFile structs from them.
var filesInfo []apexFile // 讲解注释②：apexFile=（构建产物，APEX 内路径，文件类别）三元组——
// APEX 内容清单的最小单位，后续打包/Android.mk 投影都消费它
var provideNativeLibs []string // 本 APEX 对外提供的原生库（写进 apex_manifest）
var requireNativeLibs []string // 本 APEX 需要平台提供的原生库
ctx.WalkDepsBlueprint(func(child, parent blueprint.Module) bool {
depTag := ctx.OtherModuleDependencyTag(child)
if _, ok := depTag.(android.ExcludeFromApexContentsTag); ok {
return false // 讲解注释③：依赖边上的"标签"决定文件去向——排除标签不进 APEX 也不递归
}
depName := ctx.OtherModuleName(child)
if _, isDirectDep := parent.(*apexBundle); isDirectDep {
switch depTag {
case sharedLibTag, jniLibTag:
...
return true // track transitive dependencies // 讲解注释④：返回 true=递归传递依赖——
```

```
// native_shared_libs 写一个库，它依赖的 libbase 等自动跟进
case executableTag, javaLibTag, bcpfTag, sscpfTag, shBinaryTag:
... // 讲解注释⑤：每类标签配一个 apexFileFor* 收集器（可执行/Java 库/
} // bootclasspath 片段/systemserverclasspath 片段/脚本）
}
return false
})
...（中段略：收尾校验 private key——key 属性指向的密钥模块此刻必须已解析出来，
// 否则 PropertyErrorf("key", "private_key for %q could not be found", ...)）
// Remove duplicates in filesInfo
removeDup := func(filesInfo []apexFile) []apexFile { ... }
filesInfo = removeDup(filesInfo) // 讲解注释⑥：按"APEX 内目标路径"去重——直接依赖与传递依赖
// 撞路径时视为直接依赖优先（直连/传递标记做合并）
// Sort to have consistent build rules
sort.Slice(filesInfo, func(i, j int) bool {
return filesInfo[i].path() < filesInfo[j].path()
}) // 讲解注释⑦：按目标路径排序=生成命令顺序确定——ninja 输入哈希稳定，
// 增量与可复现的基础（ch4/ch7）；源注释原话"consistent ordering even if the
// source of the files changes"
// 3) some fields in apexBundle struct are configured
a.installDir = android.PathForModuleInstall(ctx, "apex") // 讲解注释⑧：安装到
system/apex/（第四章安装目录）
a.filesInfo = filesInfo
...（suffix/payloadFsType/linkToSystemLib 配置，见 §10.4）
// 4) generate the build rules to create the APEX. This is done in builder.go.
a.buildManifest(ctx, provideNativeLibs, requireNativeLibs) // 讲解注释⑨：先产
apex_manifest.pb/json（bui
lder.go:184）
if a.properties.ApexType == flattenedApex {
a.buildFlattenedApex(ctx) // 讲解注释⑩：扁平 APEX=散装文件直装分区（免挂载，老设备过渡用）
} else {
a.buildUnflattenedApex(ctx) // 标准 APEX=apexer 打成带签名的 .apex 载荷（builder.go:381）
}
...（收尾：a.buildApexDependencyInfo/buildLintReports——依赖信息与 lint 报告）
}
```

注意第②步与第七章两遍模型的关系：WalkDepsBlueprint 能遍历，正因为阶段C已把全图与变体展开完毕——依赖边的标签（sharedLibTag/javaLibTag/...）由各模块声明 deps 时打上，APEX 只负责按标签分拣。这解释了第三章"aidl_interface 四后端"与 apex 的联动：APEX 收 java 后端时拿的是对应 APEX 变体的产物。

10.3 apexRule：一条 ninja 规则封装六件工具

第④步把实际打包委托给 builder.go。先看工具注册，再看核心规则：

go build/soong/apex/builder.go:40-70 与 96-115（原文+讲解注释）

```
pctx.HostBinToolVariable("apexer", "apexer")
// 讲解注释①：apexer——打包工具链的"总指挥"，实体是 out/host/linux-x86/bin/apexer（ch1 §1.2
host 工具）
hostBinToolVariableWithPrebuilt := func(name, prebuiltDir, tool string) { ... }
// 讲解注释②：ART 精简树（master-art manifest）没有 frameworks/base，编不出 aapt2——
// 回退 prebuilts/sdk/tools 的预编译版；"源码树完整性"成为工具选择条件
hostBinToolVariableWithPrebuilt("aapt2", "prebuilts/sdk/tools", "aapt2")
```

```
pctx.HostBinToolVariable("avbtool", "avbtool") // 讲解注释③：AVB 签名——给载荷镜像加
hashtree+footer（第十一章）
pctx.HostBinToolVariable("e2fsdroid", "e2fsdroid") // ext4
载荷制作同族：mke2fs/resize2fs/make_f2fs/mak
e_erofs
pctx.HostBinToolVariable("merge_zips", "merge_zips")
...
pctx.HostBinToolVariable("zipalign", "zipalign")
pctx.HostBinToolVariable("conv_apex_manifest", "conv_apex_manifest") // manifest json 与 pb
互转
pctx.HostBinToolVariable("apex_compression_tool", "apex_compression_tool") // 压缩
APEX（.capex）专用
...
apexRule = pctx.StaticRule("apexRule", blueprint.RuleParams{
Command: `rm -rf ${image_dir} && mkdir -p ${image_dir} && ` +
`(. ${out}.copy_commands) && ` + // 讲解注释④：rspfile 里是 buildUnflattenedApex 生成的
// cp/ln -sfn 命令序列——把各模块产物"摆"进 image_dir
`APEXER_TOOL_PATH=${tool_path} ` +
`${apexer} --force --manifest ${manifest} ` +
`--file_contexts ${file_contexts} ` + // 讲解注释⑤：SELinux 文件标签（书14 策略的文件侧）
`--canned_fs_config ${canned_fs_config} ` + // 讲解注释⑥：uid/gid/mode 清单（builder.go:1023）
`--include_build_info ` +
`--payload_type image ` +
`--key ${key} ${opt_flags} ${image_dir} ${out} `,
// 讲解注释⑦：一条规则内完成 mkfs→摆文件→SELinux 标签→AVB hashtree→签名——
// apexer 内部串联 e2fsdroid/avbtool/zipalign 等，ninja 只看到一条命令
CommandDeps: []string{"${apexer}", "${avbtool}", "${e2fsdroid}", "${merge_zips}",
"${mke2fs}", "${resize2fs}", "${sefcontext_compile}", "${make_f2fs}", "${sload_f2fs}", "
${make_erofs}",
"${soong_zip}", "${zipalign}", "${aapt2}", "prebuilts/sdk/current/public/android.jar"},
// 讲解注释⑧：CommandDeps=隐式输入——apexer 可能调用的全部工具逐一登记，
// 任一工具升级都使本规则失效重跑（ch7 §7.5 增量正确性在规则粒度的落地）
Rspfile: "${out}.copy_commands",
RspfileContent: "${copy_commands}",
Description: "APEX ${image_dir} => ${out}",
}, "tool_path", "image_dir", "copy_commands", "file_contexts", "canned_fs_config", "key",
"opt_f
lags", "manifest", "payload_fs_type")
```

设计洞察 · 逐段精读：为什么一条规则吞掉整条工具链

apexer 的输入是"一个装好文件的目录"而不是文件列表——所以 copy_commands 先把目录摆好，apexer 再对目录整体 mkfs。拆成多条 ninja 规则（先 mkfs 再逐文件 cp 再签名）会让中间目录成为共享可变状态，并发下互相踩踏；一条规则+沙箱输出（unsigned 文件名后缀 .unsigned）才是原子单元。代价是 CommandDeps 必须穷举 apexer 的全部子工具——注释里 "TODO(jiyong): use the RuleBuilder"（builder.go:562）说的就是把这套手写依赖迁移到第七章 RuleBuilder 的 sbox 机制上。

10.4 buildUnflattenedApex 与三个工程决策

builder.go:381 的 buildUnflattenedApex 分四小步（原注释 Step 1-3）：Step 1 生成 copy_commands（含符号链接优化）；Step 2 canned_fs_config（文件权限/属主编码）；Step 3 组 apexer 选项并调 apexRule。三个值得记住的决策点：

- **符号链接优化 linkToSystemLib**（apex.go:2214-2246）：捆绑 APEX 里"因传递依赖而收进来的平台可用库"不复制进载荷，改放指向 /system/lib 的符号链接——同库跨 APEX 共享，省存储。但可更新 APEX 强制关闭（源注释引 b/149805758：升级卸载 APEX 后符号链接悬空，会污染"系统健康"统计）；vendor/product 分区 APEX 也不可用（跨分区链接越界）。
- **压缩 APEX**（builder.go:551）：`compressionEnabled := ctx.Config().CompressedApex() && a.overridableProperties.Compressible && !a.testApex && ! ctx.Config().UnbundledBuildApps()`——四个条件全开才压，产物 .capex 由 apex_compression_tool 在安装时解压。默认不开：解压占安装时间，只在 /data 空间紧张的产品上点 compressible: true。
- **hashtree 裁剪**（builder.go:640 附近 optFlags `--no_hashtree`）：装在内置分区（/system 等）且不做激活校验的 APEX 可去掉 AVB hashtree 省空间——与第十一章"载荷自校验"呼应：内置分区由 vbmeta 链校验，APEX 自身 hashtree 只在独立激活路径才必需。

> **设计洞察 · 设计洞察：APEX 是"把分区做成模块"**
> 传统分区（system/vendor）粒度太粗、APK 粒度太细且缺原生库语义，APEX 用"带文件系统镜像的目录+manifest+自校验"补上中间层。构建系统的对应动作是把 ch7 的模块产物树再加一层"再打包"抽象——filesInfo 收集（本章 §10.2）到 apexer（§10.3）的流水，与第九章 target-files"一份构建多处发行"是同一思想在模块尺度的重演：**中间表示让"编译"与"交付"解耦**。

第十一章 签名体系与 AVB：testkey、releasekey 与 vbmeta

> **设计洞察 · 学习目标**
> 1. 能区分三套签名宇宙（APK signapk/APEX apexer/分区 avbtool）各自的密钥格式与裁决变量；2. 能背出 DEFAULT_SYSTEM_DEV_CERTIFICATE 的默认值与覆盖路径；3. 能解释 vbmeta.img 的描述符与链式分区（--chain_partition）机制；4. 能在排障手册场景四基础上说出"构建侧验证密钥来源"的三条命令。

11.1 链路位置：第九章签名的"密钥供给侧"

第九章 §9.5 讲了 sign_target_files_apks.py 怎么**替换**密钥；本章讲密钥体系本身——构建时谁在什么层、用什么格式、依据哪个变量选钥匙。三套宇宙互不通用：

层级	工具	密钥格式	裁决变量	产物
APK/Java	signapk	pk8+x509.pem	DEFAULT_SYSTEM_DEV_CERTIFICATE 与模块 certificate 属性	META-INF/ 签名块
APEX	apexer 内调 avbtool	同上 pk8+pem（key 属性）	apex{} 的 key 属性	apex_pubkey+载荷 footer

层级	工具	密钥格式	裁决变量	产物
分区镜像	avbtool	RSA pem	BOARD_AVB_KEY_PATH/ BOARD_AVB_ALGORITHM	hashtree+vbmeta 描述符

testkey 不是"一个密钥"而是**一套**：build/make/target/product/security/ 下 platform（核心框架）、shared、media、bluetooth、testkey 五对 pk8/pem（README 明确列出生成命令 development/tools/make_key）。APK 的 certificate 属性值如 `platform` 即指同名密钥对——签名决定的是 Android 权限域（platform 签名才能拿到 signature 级权限），不只是完整性。

11.2 默认密钥裁决链

APK 侧与 vbmeta 侧的默认值落在两个文件里，合起来看：

mk build/make/core/config.mk:795-800 与 build/make/core/Makefile:4006-4014（原文+讲解注释）

```
ifdef PRODUCT_DEFAULT_DEV_CERTIFICATE
DEFAULT_SYSTEM_DEV_CERTIFICATE := $(PRODUCT_DEFAULT_DEV_CERTIFICATE)
# 讲解注释①：产品级覆盖——device.mk 里 PRODUCT_DEFAULT_DEV_CERTIFICATE 指向自家密钥对
else
DEFAULT_SYSTEM_DEV_CERTIFICATE := build/make/target/product/security/testkey
# 讲解注释②：无任何配置落回 testkey——"userdebug 出厂即 testkey"的出处，
# 排障手册场景四 INSTALL_PARSE_FAILED_INCONSISTENT_CERTIFICATES 的构建侧根源
endif
.KATI_READONLY := DEFAULT_SYSTEM_DEV_CERTIFICATE
# 讲解注释③：只读锁（ch8 记账思想的孪生：关键变量一次性定死，防后续 .mk 暗改）
# ---- 以下为 Makefile 侧（BOARD_AVB_ENABLE=true 时的 vbmeta 默认值）----
ifdef BOARD_AVB_KEY_PATH
$(if $(BOARD_AVB_ALGORITHM),,$(error BOARD_AVB_ALGORITHM is not defined))
# 讲解注释④：给了密钥就必须给算法——BoardConfig.mk 两变量成对，缺一即 error
else
# If key path isn't specified, use the 4096-bit test key.
BOARD_AVB_ALGORITHM := SHA256_RSA4096
BOARD_AVB_KEY_PATH := external/avb/test/data/testkey_rsa4096.pem
# 讲解注释⑤：AVB 默认密钥不在 security/ 目录，而在 external/avb——
# pk8/pem 宇宙（signapk）与 RSA pem 宇宙（avbtool）是两套独立密钥库
endif
```

Makefile:730-734 再把 APK 默认密钥提升为发行物级：注释原文 "The dev key is used to sign this package, and as the key required for future OTA packages installed by this system"，随后 `DEFAULT_KEY_CERT_PAIR := $(strip $(DEFAULT_SYSTEM_DEV_CERTIFICATE))`——第九章 §9.4 otapackage 规则里 `-k $(KEY_CERT_PAIR)` 的值就是它。

11.3 APK 签名规则：soong 侧的 signapk

模块的产物签名发生在 soong 阶段（java/app_builder.go）——APK 在 .intermediates 里就已经是签好名的：

go build/soong/java/app_builder.go:33-41 与 113-121（原文+讲解注释）

```
Signapk, SignapkRE = pctx.RemoteStaticRules("signapk",
blueprint.RuleParams{
```

```
Command: `rm -f $out && $reTemplate${config.JavaCmd} ${config.JavaVmFlags} -Djava.librar
y.path=$$(dirname ${config.SignapkJniLibrary}) ` +
`-jar ${config.SignapkCmd} $flags $certificates $in $out`,
CommandDeps: []string{"${config.SignapkCmd}", "${config.SignapkJniLibrary}"},
},
&remoteexec.REParams{Labels: map[string]string{"type": "tool", "name": "signapk"}, ...},
[]string{"flags", "certificates"}, []string{"implicits", "outCommaList"})
// 讲解注释①：signapk 是 Java 工具（-jar），还需 JNI 库做 PKCS 加密——所以命令里带
// -Djava.library.path；RemoteStaticRules 说明它可上 RBE 远程执行（第十三章）
...
if ctx.Config().UseRBE() && ctx.Config().IsEnvTrue("RBE_SIGNAPK") {
rule = SignapkRE // 讲解注释②：远程执行还要显式开 RBE_SIGNAPK——私钥不出本机的保守默认
...
}
ctx.Build(pctx, android.BuildParams{
Rule: rule,
Description: "signapk",
Outputs: outputFiles,
Input: unsignedApk, // 讲解注释③：先出 -unsigned.apk 再签——签名是独立 ninja 边，
Implicits: deps, // 换密钥只重签不重编（增量友好，ch4 思想）
Args: args,
})
```

certificate 属性的解析在 java 包的 getCertificateString 处："platform"/"shared"/"media" 等名字映射到 DEFAULT_SYSTEM_DEV_CERTIFICATE 所在目录下的同名密钥对；apexBundle 的 getCertString（apex.go:1405）同理。所以**默认密钥目录一换，全树签名跟着换**——这就是 sign_target_files_apks 的 -e 映射能在发行链路整体重签的前提。

11.4 vbmeta.img：信任根的"目录页"

分区签名不是"把镜像整体签一遍"，而是 AVB 的描述符模型：每个分区镜像尾部有 hashtree+footer，vbmeta.img 收纳各分区的**哈希描述符**并用顶层密钥签名。链式分区再下一层——vbmeta_system.img 自己被签后，顶层 vbmeta 只存它的公钥：

mk build/make/core/Makefile:4214-4245 与 4494-4517（骨架+讲解注释）

```
# $(1): the partition to enable AVB chain, e.g., boot or system or vbmeta_system.
define _check-and-set-avb-chain-args
$(eval part := $(1))
$(eval PART=$(call to-upper,$(part)))
$(eval _key_path := BOARD_AVB_$(PART)_KEY_PATH)
$(eval _signing_algorithm := BOARD_AVB_$(PART)_ALGORITHM)
$(eval _rollback_index := BOARD_AVB_$(PART)_ROLLBACK_INDEX)
$(eval _rollback_index_location := BOARD_AVB_$(PART)_ROLLBACK_INDEX_LOCATION)
$(if $($(_key_path)),,$(error $(_key_path) is not defined))
# 讲解注释①：链式分区四件套（key/algorithm/rollback index/location）缺一即 error
...
$(eval $(_signing_args) := \
--algorithm $($(_signing_algorithm)) --key $($(_key_path)))
...
$(eval INTERNAL_AVB_MAKE_VBMETA_IMAGE_ARGS += \
--chain_partition $(part):$($(_rollback_index_location)):$(AVB_CHAIN_KEY_DIR)/$(part).avbpub
key))
```

```
# 讲解注释②：链式分区的核心——顶层 vbmeta 只记"子 vbmeta 公钥+回滚槽位"，
# 描述符本体在 vbmeta_system.img/vbmeta_vendor.img；公钥由 extract-avb-chain-public-keys 预提取
...
endef
define build-vbmetaimage-target
$(call pretty,"Target vbmeta image: $(INSTALLED_VBMETAIMAGE_TARGET)")
$(hide) mkdir -p $(AVB_CHAIN_KEY_DIR)
$(call extract-avb-chain-public-keys, $(AVB_CHAIN_KEY_DIR))
# 讲解注释③：Makefile:4390-4432 逐分区提取公钥（boot/init_boot/vendor_boot/system/vendor/
# product/system_ext/odm/各 dlkm...），产出 AVB_CHAIN_KEY_DIR/<part>.avbpubkey
$(hide) $(AVBTOOL) make_vbmeta_image \
$(INTERNAL_AVB_MAKE_VBMETA_IMAGE_ARGS) \
$(PRIVATE_AVB_VBMETA_SIGNING_ARGS) \
$(BOARD_AVB_MAKE_VBMETA_IMAGE_ARGS) \
--output $@
# 讲解注释④：顶层 vbmeta.img=一次 make_vbmeta_image——本身不含数据，
# 只有描述符与链式公钥；签名密钥即 §11.2 的 BOARD_AVB_KEY_PATH
$(hide) rm -rf $(AVB_CHAIN_KEY_DIR)
endef
$(INSTALLED_VBMETAIMAGE_TARGET): \
$(AVBTOOL) \
$(INSTALLED_BOOTIMAGE_TARGET) \
$(INSTALLED_INIT_BOOT_IMAGE_TARGET) \
...
$(INSTALLED_SYSTEMIMAGE_TARGET) \
$(INSTALLED_VENDORIMAGE_TARGET) \
...
# 讲解注释⑤：依赖边=全部（非链式）分区镜像——任一分区重打，vbmeta 必须重签。
# 它是设备完整性的"目录页"：bootloader 校验 vbmeta，vbmeta 描述符再校验各分区
```

回滚防护也在这一层：BOARD_AVB_<PART>_ROLLBACK_INDEX 写进 footer，bootloader 拒绝 index 低于本机的镜像——与第九章 §9.5 OTA metadata 的分区时间戳防线是两条独立锁链。

> **设计洞察 · 设计洞察：为什么 vbmeta 单独成镜像**
>
> 把签名信息从数据里抽出来，换来三件事：1. 顶层校验恒定——bootloader 只需认一个固定位置的小文件，分区增删只动描述符；2. 防回滚与密钥轮换有独立槽位（--chain_partition 的 rollback_index_location）；3. 签名与打镜像解耦——第十章 apexer 对载荷做同样的事（apex_pubkey+footer），第九章 add_img_to_target_files 对 target-files 做同样的事。**"完整性目录"与"完整性数据"分离**是 AVB 2.0 相对传统 verity 的本质演进。

11.5 构建侧验证密钥来源：三条命令

面试与排障共用（衔接排障手册场景四）：

- 1. `grep PRODUCT_DEFAULT_DEV_CERTIFICATE out/soong/make_vars-<product>.mk`——soong 导出给 Kati 的变量合同（第九章 §9.2 三文件之一）里看产品实际落点；
- 2. `unzip -p out/target/product/<device>/*-target_files-*.zip META/otakeys.txt`——OTA 侧密钥（第九章 ReplaceOtaKeys 的对象）；

- 3. `avbtool info_image --image out/target/product/<device>/vbmeta.img`——直接读 vbmeta 描述符：算法、公钥指纹、链式分区、rollback index 一次看全。

> **设计洞察 · 逐段精读：release-keys 从哪来**
> 构建全程只产 testkey（或产品自配密钥）版本；`ro.build.tags=release-keys` 的镜像全部出自 sign_target_files_apks.py 的重签链路（第九章 §9.5：-e 映射逐 APK 重签、ReplaceCerts 替换策略内指纹、ReplaceOtaKeys 重写 otakeys.txt，最后回读 vbmeta 也要 -k 换 AVB 密钥）。所以"拿到 release 包"的正确姿势是**拿签名版 target-files 再跑 ota_from_target_files**，而不是给构建机塞 release 密钥——后者让私钥暴露在构建网段，违背密钥分层（书7 面试题"为什么 release 密钥不在源码树"的机理答案）。

第十二章 遗留 Android.mk 的生存边界与 androidmk

> **设计洞察 · 学习目标**
> 1. 能说出 Android 13 树上存量 Android.mk 的规模分布与"引擎 .mk/模块 .mk"两类归属；2. 能走读 androidmk 的三段流水线（mkparser→bpFile→bpfix）并指出它的白名单本质；3. 能列举 .mk 中"不该迁移"的合法残存物类型；4. 能用 BUILD_BROKEN_* 族把严格性网与逃生门讲成一套制度。

12.1 存量盘点：谁还留在 Make 世界

第五章说"Kati 保留是因为数百万行 Make 规则迁不完"；本章把这句话数字化。模块类 Android.mk（定义目标的）在核心仓库已基本清零：本树 frameworks/ 与 packages/ 合计仅存 74 个，且多为复制声明、docs 目标这类"非模块逻辑"（§12.5）。而 build/make/ 下还有 359 个 .mk——它们不是遗留，是**构建引擎自身**（main.mk/config.mk/product_config.mk，第六至九章的主角）。区分"引擎的 mk"与"模块的 mk"是理解边界的第一个动作；第二章 device 树里的 BoardConfig.mk 属于前者（第五章设计决策：板级配置是硬件事实）。

12.2 androidmk：三段流水线

androidmk 是自动化迁移工具（build/soong/androidmk/，入口 cmd/androidmk.go:35 的 usage：解析 Android.mk "并尝试输出类似的 Android.bp 到标准输出"）。核心在 ConvertFile：

go build/soong/androidmk/androidmk/androidmk.go:132-186（原文+讲解注释）

```
func ConvertFile(filename string, buffer *bytes.Buffer) (string, []error) {
p := mkparser.NewParser(filename, buffer)
nodes, errs := p.Parse()
if len(errs) > 0 {
return "", errs
}
// 讲解注释①：第一段——mkparser 把 .mk 文本解析成节点流（Assignment/Directive/Comment
// 三类），只认 make 语法不认 Android 语义
file := &bpFile{scope: androidScope(), ...}
// 讲解注释②：androidScope() 是"词汇表"——my-dir/all-java-files-under 的 Go 仿真与
// BUILD_* → bp 类型映射（android.go:1008-1040，见 §12.3）；bpFile 另有变量表与
```

```
// conds/assignmentCond/tree 三个状态变量（此处从略）
for _, node := range nodes {
file.setMkPos(p.Unpack(node.Pos()), p.Unpack(node.End()))
switch x := node.(type) {
case *mkparser.Comment:
file.insertComment("//" + chunks[0])
// 讲解注释③：注释原样保留为 // 注释——生成的 bp 保留原文可读性
case *mkparser.Assignment:
handleAssignment(file, x, assignmentCond)
// 讲解注释④：LOCAL_SRC_FILES→srcs、LOCAL_MODULE→name 逐变量翻译在此
case *mkparser.Directive:
switch x.Name {
case "include", "-include":
module, ok := mapIncludePath(x.Args.Value(file.scope))
if !ok {
file.errorf(x, "unsupported include")
// 讲解注释⑤：include 只认白名单（android.go:1000）——androidmk 是翻译器
// 不是通用 Make 解释器
continue
}
switch module {
case clearVarsPath:
resetModule(file) // 讲解注释⑥：include $(CLEAR_VARS)=模块边界——开启新模块
case includeIgnoredPath:
// subdirs are already automatically included in Soong
continue // 讲解注释⑦：all-subdir-makefiles 直接忽略——Soong 自动扫目录
default:
makeModule(file, module) // 讲解注释⑧：include $(BUILD_SHARED_LIBRARY)=收尾落模块
}
case "ifeq", "ifneq", "ifdef", "ifndef", "else", "endif":
... // 讲解注释⑨：条件只翻 conditionalTranslations 白名单；其余记
// "unsupported conditional" 留人工，else/endif 维护条件栈
default:
file.errorf(x, "unsupported directive")
continue
}
default:
file.errorf(x, "unsupported line")
}
}
tree = &bpparser.File{Defs: file.defs, Comments: file.comments}
// 讲解注释③b：节点流遍历完毕——语义对象装进 bp 语法树，进入后处理
// check for common supported but undesirable structures and clean them up
fixer := bpfix.NewFixer(tree)
fixedTree, fixerErr := fixer.Fix(bpfix.NewFixRequest().AddAll())
// 讲解注释⑩：第三段 bpfix 后处理——变量内联、属性清洗、中间类型改写
out, err := bpparser.Print(tree) // 讲解注释⑪：按标准格式输出 Android.bp 到 stdout
return string(out), errs
}
```

三段式与第七章 soong_build 同构：解析（不求值）→语义映射→格式化输出。区别在 soong_build 的输出是 ninja 动作，androidmk 的输出是新的声明式源码——**迁移工具把命令式 .mk 编译成声明式 .bp**。

12.3 翻译的词汇表：BUILD_* 映射与函数仿真

go build/soong/androidmk/androidmk/android.go:959-977 与 1008-1020（原文+讲解注释）

```
"BUILD_SHARED_LIBRARY": "cc_library_shared",
"BUILD_STATIC_LIBRARY": "cc_library_static",
...
"BUILD_EXECUTABLE": "cc_binary",
...
"BUILD_JAVA_LIBRARY": "java_library_installable", // will be rewritten to java_libra
ry by bpfix
...
"BUILD_PACKAGE": "android_app",
// 讲解注释①：BUILD_* include 变量→bp 模块类型的词典是 androidmk 的词法核心；
// 与第三章模块类型速查表一一对应。*_installable 是中间形态，由 bpfix 清洗
// （讲解注释⑩），产物 bp 里看不到它
...
func androidScope() mkparser.Scope {
globalScope := mkparser.NewScope(nil)
globalScope.Set("CLEAR_VARS", clearVarsPath)
globalScope.SetFunc("my-dir", mydir)
globalScope.SetFunc("all-java-files-under", allFilesUnder("*.java"))
// 讲解注释②：make 函数受限仿真——$(call my-dir)/$(call all-java-files-under) 有语义等价物；
// 而 all-subdir-makefiles/all-named-subdir-makefiles 映射到
includeIgnored（android.go:1021-1025），
// 即"目录递归 include"在 Soong 世界无对应物，直接由自动扫描取代
```

翻译失败的形态也有固定样貌——errorf（androidmk.go:80-90）把失败节点原文 Dump 出来，以 `// ANDROIDMK TRANSLATION ERROR: ...` 注释插进输出的 bp：产物**可读但不可编**，人工按注释逐条修。这是"机器尽力、人管例外"的务实分工。

12.4 .mk 的合法残存物：frameworks/base/Android.mk 剖面

为什么 74 个 .mk 迁不走？看 frameworks/base/Android.mk（全文约 120 行，无一个模块定义）：

- `$(eval $(call declare-1p-copy-files,frameworks/base,.ogg))` 系列——向 Kati 声明第一方复制文件清单（铃声/键位布局/IDC），这是 build/make 引擎的钩子而非 Soong 模块；
- `include $(LOCAL_PATH)/pathmap.mk`——加载框架内目录映射表，供老依赖方查询；
- docs/offline-sdk-docs 等纯目标——文档流水线，没有 bp 模块类型对应物。

它们共同点：**不是"模块怎么编译"而是"引擎的附加声明与边角目标"**——正好落在第九章 §9.2 的边界合同之外（不进 Android-<product>.mk 的模块翻译账本，而由 Android.mk.list 直通 main.mk）。第五册附录"加新系统 App"类操作全在 bp 世界；碰到这三类需求才需要回 mk。

12.5 严格性网与逃生门：BUILD_BROKEN 族

迁移期最大的工程风险不是"翻不动"而是"放任坏味道"。Android 13 的制度是**每类坏味道一个具名开关**（board_config.mk:176-198）：

mk build/make/core/board_config.mk:176-198 与 209（原文+讲解注释）

```
_build_broken_var_list := \
BUILD_BROKEN_DUP_RULES \
BUILD_BROKEN_DUP_SYSPROP \
BUILD_BROKEN_ELF_PREBUILT_PRODUCT_COPY_FILES \
BUILD_BROKEN_ENFORCE_SYSPROP_OWNER \
BUILD_BROKEN_INPUT_DIR_MODULES \
BUILD_BROKEN_MISSING_REQUIRED_MODULES \
BUILD_BROKEN_OUTSIDE_INCLUDE_DIRS \
BUILD_BROKEN_PREBUILT_ELF_FILES \
BUILD_BROKEN_TREBLE_SYSPROP_NEVERALLOW \
BUILD_BROKEN_USES_NETWORK \
BUILD_BROKEN_VENDOR_PROPERTY_NAMESPACE \
BUILD_BROKEN_VINTF_PRODUCT_COPY_FILES \
# 讲解注释①：十二类具名坏味道——重复定义规则/重复 sysprop/预编译 ELF 直拷/
# 构建期联网……每项对应一处检查点；置 true=本板显式豁免该检查
$(eval _build_broken_var_list += $(foreach m,...,BUILD_BROKEN_USES_$(m)))
# 讲解注释②：还按"模块类型可用性"自动展开 BUILD_BROKEN_USES_<type>——
# 每个被标记为即将禁用的模块类型都有专属逃生门
...
BUILD_BROKEN_NINJA_USES_ENV_VARS :=
# 讲解注释③：第 209 行——ninja 环境白名单的逃生门（对应习题 15）单独管理，
# 不进 _board_true_false_vars 列表（它由 soong_ui 的 ninja.go 消费而非 board_config 校验）
```

与 kati 的 `--werror_overriding_commands` 联动（kati.go:204-208：`if ! config.BuildBrokenDupRules() { args = append(args, "--werror_overriding_commands") }`）——检查点在 ckati 里，开关经 soong_ui 传参豁免。第九章 §9.1 的 `--werror_implicit_rules`/`--werror_real_to_phony` 同族但无逃生门（隐式规则、phony 混用零容忍）。

设计洞察 · 设计洞察：技术债要"具名"不要"泛化"

BUILD_BROKEN 族的精髓是**把每个坏味道登记成显式变量**，而不是一个 BUILD_NO_STRICT 总闸。于是：CI 能按项统计"哪块板还欠哪笔债"；新检查上线默认全开，迁移成本压到"改动一行 BoardConfig"而不是"关掉全部严格性"；逃生日志天然成为迁移清单。第五章"声明式可静态校验"的决策，靠这套制度才落得了地——**严格性先于完备性**。

第十三章 构建性能工程：并发、内存池与 RBE

设计洞察 · 学习目标

1. 能推导 parallel=NumCPU+2 与 highmem_pool 深度的计算公式并说出每档内存对应的并发数；2. 能用 NINJA_HIGHMEM_NUM_JOBS/NINJA_REMOTE_NUM_JOBS/NINJA_ARGS 三个旋钮应对不同瓶颈；3. 能解释 ninja 环境白名单存在的原因及其逃生门；4. 能描述 RBE reproxy 引导流程与远程池 500 并发的容量模型。

13.1 链路位置：第四章增量的"执行侧"补全

第四章的增量机制回答"哪些不用编"；本章回答"要编的怎么跑得快"。两条主线的物理入口都汇合在第六章阶段五 runNinjaForBuild：ninja 拿着 combined.ninja 按 -j 并发调度，动作落在本地 CPU 池、高内存池或远程执行池三个容量域。性能工程的三层杠杆——单机并发（§13.2）、池化隔离（§13.3）、远程卸载（§13.4）——分别对应 config.go 的三段代码。

13.2 单机并发：NumCPU+2 与内存分级

go build/soong/ui/build/config.go:236-247 与 1041-1064 与 1230-1240（骨架+讲解注释）

```
func NewConfig(ctx Context, args ...string) Config {
ret := &configImpl{
environ: OsEnvironment(),
sandboxConfig: &SandboxConfig{},
}
// Default matching ninja
ret.parallel = runtime.NumCPU() + 2
// 讲解注释①：总并发默认=核数+2——与 ninja 官方默认一致；+2 让编译间隙的
// 串行动作（链接/打包/签名）填进核间空隙
ret.keepGoing = 1
// 讲解注释②：-k 1=遇错即收尾——排障手册场景一"屏幕最后一屏不是第一个错误"的根源
ret.totalRAM = detectTotalRAM(ctx)
// 讲解注释③：内存总量探测一次，供 HighmemParallel 分级决策（④-⑨）
...
func (c *configImpl) HighmemParallel() int {
if i, ok := c.environ.GetInt("NINJA_HIGHMEM_NUM_JOBS"); ok {
return i
}
// 讲解注释④：环境变量覆盖入口——排障手册场景三的手动限流开关
const minMemPerHighmemProcess = 8 * 1024 * 1024 * 1024
// 讲解注释⑤：每个 highmem 动作（dex2oat/d8）按 8GB 内存预算计算
parallel := c.Parallel()
if c.UseRemoteBuild() {
// Return 1/16th of the size of the local pool, rounding up.
return (parallel + 15) / 16
// 讲解注释⑥：远程构建时本地只做调度与收尾——highmem 取总并发 1/16 向上取整，
// 源注释明说原因：ninja 不支持嵌套池，两池相加会超订 CPU
} else if c.totalRAM <= 16*1024*1024*1024 {
return 1 // 讲解注释⑦：≤16GB 机器只允许 1 个 highmem 并发
} else if c.totalRAM <= 32*1024*1024*1024 {
return 2 // 讲解注释⑧：≤32GB 允许 2 个
} else if p := int(c.totalRAM / minMemPerHighmemProcess); p < parallel {
return p // 讲解注释⑨：更大内存按 8GB/进程向下取整，且不超过总并发
}
return parallel
}
func (c *configImpl) RemoteParallel() int {
if !c.UseRemoteBuild() {
return 0
}
if i, ok := c.environ.GetInt("NINJA_REMOTE_NUM_JOBS"); ok {
return i
```

```
}
return 500
// 讲解注释⑩：远程池默认 500——与本地池是两个独立容量维度，见 §13.4
}
```

这组数字解释了排障手册场景三的机制细节：16GB 开发机上 dex2oat 高峰只有一个进程、32GB 两个——不是玄学而是 config.go:1056-1064 的分支表；`NINJA_HIGHMEM_NUM_JOBS=2` 能起作用则因为分支表第一行先查环境变量。

13.3 ninja 侧参数注入与环境白名单

soong_ui 拼装 ninja 命令行与运行环境的完整逻辑（ninja.go:57-135）：

go build/soong/ui/build/ninja.go:57-89 与 101-135（原文+讲解注释）

```
args = append(args, "-j", strconv.Itoa(parallel))
if config.keepGoing != 1 {
args = append(args, "-k", strconv.Itoa(config.keepGoing))
}
// 讲解注释①：-j/-k 从 config 来——§13.2 的两个数字在此落进命令行
args = append(args, "-f", config.CombinedNinjaFile())
// 讲解注释②：唯一入口=combined.ninja（第六章 §6.3）
args = append(args,
"-o", "usesphonyoutputs=yes",
"-w", "dupbuild=err",
"-w", "missingdepfile=err")
// 讲解注释③：三处严格性告警升级为错误——phony 冲突/重复构建/缺 depfile 零容忍
cmd := Command(ctx, config, "ninja", executable, args...)
// Set up the nsjail sandbox Ninja runs in.
cmd.Sandbox = ninjaSandbox
// 讲解注释④：ninja 自身跑在 nsjail 沙箱里（第七章 sbox 的进程级兄弟）
if config.HasKatiSuffix() {
// Reads and executes a shell script from Kati that sets/unsets the
// environment Ninja runs in.
cmd.Environment.AppendFromKati(config.KatiEnvFile())
// 讲解注释⑤：kati 生成的 env 脚本在 ninja 启动前执行——kati 与 ninja 的环境对齐
}
// Allow both NINJA_ARGS and NINJA_EXTRA_ARGS, since both have been
// used in the past to specify extra ninja arguments.
if extra, ok := cmd.Environment.Get("NINJA_ARGS"); ok {
cmd.Args = append(cmd.Args, strings.Fields(extra)...)
}
if extra, ok := cmd.Environment.Get("NINJA_EXTRA_ARGS"); ok {
cmd.Args = append(cmd.Args, strings.Fields(extra)...)
}
// 讲解注释⑥：NINJA_ARGS/NINJA_EXTRA_ARGS 追加进命令行——
// 排障时 -d explain（查"为什么重编"）或 --debug=stat 的注入点
// Filter the environment, as ninja does not rebuild files when environment
// variables change.
//
// Anything listed here must not change the output of rules/actions when the
// value changes, otherwise incremental builds may be unsafe.
// 讲解注释⑦：白名单存在的原因——ninja 不因 env 变化重跑动作，所以凡放行的
// 变量必须"值变不出产物差异"；这正是习题 15 的机制面
```

```
if cmd.Environment.IsEnvTrue("ALLOW_NINJA_ENV") {
ctx.Println("Allowing all environment variables during ninja; incremental builds may be unsa
fe.")
} else {
cmd.Environment.Allow(append([]string{
"ASAN_SYMBOLIZER_PATH",
...（无害通行组：HOME/JAVA_HOME/LANG/LC_MESSAGES/OUT_DIR/PATH/PWD/
// PYTHONDONTWRITEBYTECODE/TMPDIR/USER 依次登记）
"TARGET_BUILD_APPS",
"TARGET_BUILD_VARIANT",
...
}))
// 讲解注释⑧：白名单三类：无害的（HOME/PATH）、sandbox 所需（TMPDIR）、
// 已被指纹机制另行追踪的（TARGET_BUILD_*——ch8 的 --regen 与 previous_build_config 兜底）
}
```

13.4 RBE：把动作卸载到远程池

远端执行不是把 ninja 换掉，而是在动作粒度上多一条容量通道。第七章 RuleBuilder 的 remoteable 字段（§7.5）声明哪些规则可上远端，signapk 的 RemoteStaticRules（第十一章 §11.3）是另一个实例；soong_ui 侧负责引导与关闭 reproxy 代理进程：

go build/soong/ui/build/rbe.go:103-127（原文+讲解注释）

```
func startRBE(ctx Context, config Config) {
ctx.BeginTrace(metrics.RunSetupTool, "rbe_bootstrap")
defer ctx.EndTrace()
if u := ulimitOrFatal(ctx, config, "-u"); u < rbeLeastNProcs {
ctx.Fatalf("max user processes is insufficient: %d; want >= %d.\n", u, rbeLeastNProcs)
}
if n := ulimitOrFatal(ctx, config, "-n"); n < rbeLeastNFiles {
ctx.Fatalf("max open files is insufficient: %d; want >= %d.\n", n, rbeLeastNFiles)
}
// 讲解注释①：引导前先验 ulimit——500 路并发的远程池要海量 fd 与进程配额，
// 配额不足直接 Fatal 而不是跑到一半炸
cmd := Command(ctx, config, "startRBE bootstrap", rbeCommand(ctx, config, bootstrapCmd))
if output, err := cmd.CombinedOutput(); err != nil {
ctx.Fatalf("Unable to start RBE reproxy\nFAILED: RBE bootstrap failed with: %v\n%s\n", err,
output)
}
// 讲解注释②：reproxy 是本地代理进程——ninja 的动作发给它，它再向远端集群
// 提交/取回产物与缓存；认证配置经 rbeAuth() 注入（rbe.go:98）
}
func stopRBE(ctx Context, config Config) {
cmd := Command(ctx, config, "stopRBE bootstrap", rbeCommand(ctx, config, bootstrapCmd),
"-shutdo
wn")
...
// 讲解注释③：构建收尾 -shutdown 关代理并落指标（DumpRBEMetrics 出 protobuf）——
// 远程命中率/排队时长可离线分析，是性能回归的观测面
}
```

远程池的前提之一是动作不碰网：board_config.mk:181 的 BUILD_BROKEN_USES_NETWORK 是构建期联网动作的逃生门（第十二章 §12.5）——可上远端的动作必须可重放、无本地隐式状态，

这条纪律把"哪些动作能 remoteable"从架构上圈定了。

13.5 调优旋钮速查

旋钮	作用域	默认	典型用法
NINJA_HIGHMEM_NUM_JOBS	高内存池深度	按内存分级（§13.2）	16GB 机 dex2oat OOM 时限 1（场景三）
NINJA_REMOTE_NUM_JOBS	远程池并发	500	远端配额受限时下调
NINJA_ARGS/NINJA_EXTRA_ARGS	追加 ninja 命令行	空	注入 -d explain 排查重编
NINJA_HEARTBEAT_INTERVAL	卡死检测心跳	5 分钟（ninja.go:91）	长链接动作被误判时延长
ALLOW_NINJA_ENV	ninja 环境白名单	关闭	临时放行后必须回退（增量不安全）
RBE_SIGNAPK	signapk 上远端	关闭	私钥不出本机，默认保守

> **设计洞察 · 设计洞察：三级容量模型**
> 本地 CPU 池（NumCPU+2）、高内存池（内存/8GB）、远程池（500）是三个独立预算而非一个总开关——因为瓶颈维度不同：编译吃核、dex2oat 吃内存、RBE 吃集群配额。ninja 的 pool 机制（第六章 highmem_pool）+ 每规则 remoteable 声明，把"哪类动作消耗哪类资源"编码进构建图本身。通用经验：**调优先认瓶颈维度，再找对应池子的旋钮**——对着 CPU 池调 -j 救不了 dex2oat OOM，这正是场景三修复项分开两条命令的原因。

教科书习题集（分级）

第一级：简答题（检验概念理解）

- **1. `m` 命令的本质是什么？从敲下回车到编译执行经过哪些进程？**

  参考答案要点：1. m 是 envsetup.sh 里的函数，最终调 soong_ui.bash；2. Microfactory 自举出 Go 版 soong_ui；3. soong_ui 按阶段调 ckati/soong_build/ninja——它本身不解析任何 mk/bp（build/soong/ui/build/build.go:188）。

- **2. soong_ui 的五个阶段位是什么？各产出什么文件？**

  参考答案要点：1. RunProductConfig=2/RunSoong=4/RunKati=8/RunKatiNinja=16/RunNinja=32（build.go:94-108）；2. 依次产出 TARGET_DEVICE 等变量、out/soong/build.ninja、out/build-<product>.ninja、combined 内 subninja 行、全部编译产物。

- **3. combined-<product>-<variant>.ninja 是怎么"并线"的？subninja 与 include 的区别？**

参考答案要点：1. Go template 写出 builddir+highmem_pool+三条 subninja（kati build/kati package/soong，build.go:61-70）；2. subninja 作用域隔离——两个引擎的同名 rule/variable 互不污染，include 是文本拼接作用域共享。

- **4. inherit-product 与直接 include 父 makefile 的本质区别？**

参考答案要点：1. inherit-product 给每个产品变量追加 "@inherit:路径" 占位符并记录 INHERITS_FROM（product.mk:395-409）；2. 继承变成数据：DAG 可先建后展开（node_fns.mk），钻石继承可去重、可检环；include 是立即执行的副作用。

- **5. Kati 的 --regen 模式解决什么问题？**

参考答案要点：1. 判定 CLI 参数、.mk 时间戳、env 变量、$(wildcard)/部分 $(shell) 结果是否变化（kati.go:93-95 注释）；2. 未变则不重新生成 ninja 文件，把阶段三降到秒级；3. 这是"改一行 Java 不重跑 kati 全量"的前提。

- **6. soong_build 的 RunBlueprint 分哪五步？**

参考答案要点：1. ListModulePaths→ParseFileList→ResolveDependencies→PrepareBuildActions→WriteNinja（build/blueprint/bootstrap/command.go:47-140）；2. 第三步跑 mutator 展开变体，第四步逐模块/单例生成动作，第五步写 build.ninja 与 .d。

- **7. ModuleBase.GenerateBuildActions 与模块自己的 GenerateAndroidBuildActions 是什么关系？**

参考答案要点：1. 前者是框架钩子：安装依赖收集、许可元数据、模块名后缀、可见性校验、init_rc/vintf 打包（module.go:2186-2262）；2. 收尾才调后者——模块作者只实现"怎么编译"。

- **8. ninja 的 restat 是什么？sbox 的 --write-if-changed 为什么必要？**

参考答案要点：1. restat 让重跑后输出内容未变的规则被视为"未过期"，下游不重编（rule_builder.go:103-105）；2. sbox 原子提交输出并按内容比较跳过 mtime 更新（634-638 行）——两者合起来消除代码生成器引发的重编风暴。

- **9. 切 lunch 后为什么要重跑 kati 且自动 installclean？判定文件在哪？**

参考答案要点：1. katiSuffix="-"+产品名（kati.go:43-48），combined/build ninja 全套换名，必重生成；2. TARGET_PRODUCT 变化命中 --regen 的 env 指纹；3. previous_build_config.mk 记录 product-variant，变化即 installclean（cleanbuild.go:198-250），DISABLE_AUTO_INSTALLCLEAN 可关。

- **10. target-files 与 OTA 包是什么关系？各由谁生成？**

参考答案要点：1. target-files 是未签名中间发行物（BUILT_TARGET_FILES_PACKAGE，add_img_to_target_files.py:758 组装）；2. m otapackage 的 mk 规则调 ota_from_target_files.py 吃它产出 <product>-ota-<tag>.zip（Makefile:6107-6150）；3. 正式包先经 sign_target_files_apks.py 签名再重打包。

第二级：分析题（检验机制理解）

- **11. 画出 `m services` 从敲命令到 services.jar 进 system.img 暂存区的阶段穿越图，标注每阶段产物文件与失败点。**

  参考答案要点：1. 阶段一 dump 变量→阶段二 soong_build 重分析（若 bp 变）→阶段三 kati --regen 快速通过→阶段五 ninja 从 .intermediates 增量编 services 及下游→安装到 staging；2. 失败点分布：阶段一报产品错、阶段二报 bp 错、阶段五报编译错（out/error.log）。

- **12. 一个 cc_binary 为什么会展开成几十个 ninja 目标？展开发生在 RunBlueprint 的哪一步？**

  参考答案要点：1. mutator 管道按 arch（arm64/arm/x86_64…）×os（android/vendor/host…）×link（shared/static）×APEX 轴克隆模块实例；2. 发生在 ResolveDependencies（阶段C）；3. 第七章 suffix 逻辑（module.go:2215-2225）是变体命名的物证。

- **13. 钻石继承下（子产品两次 inherit 同一祖父）PRODUCT_PACKAGES 会重复吗？逐层说清去重机制。**

  参考答案要点：1. _import-nodes-inner 的 .seen 守卫保证父节点只展开一次（node_fns.mk:234-247）；2. 展开时占位符 uniq-word 去重、父值清空防 DAG 重复（node_fns.mk:164-180）；3. 最终 main.mk 还对 ALL_MODULES 排序去重。

- **14. 若子产品已设置 PRODUCT_NAME（单值变量），_expand-inherited-values 如何裁决父值?**

  参考答案要点：1. 单值变量且子已赋值时父值被丢弃（"子赢"，node_fns.mk:158-162 的 _eiv_sv 分支）；2. 列表变量则原位拼接、顺序=inherit-product 调用位置；3. 这就是"属性覆盖优先级"的机械实现。

- **15. 为什么 ninja 只允许看到白名单里的环境变量？若某个规则确实依赖 env 该怎么办?**

  参考答案要点：1. ninja 不因 env 变化重跑动作（ninja.go:100-160 的注释与过滤清单）；2. 依赖 env 的动作会做错增量——变量变了动作不重跑；3. 逃生通道 BUILD_BROKEN_NINJA_USES_ENV_VARS（ninja.go:157），或由 makefiles/soong 把 env 显式复制进命令行。

- **16. 为什么 soong 用"先建全图、再生成动作"的两遍模型，而不是边解析边生成?**

  参考答案要点：1. 声明式依赖在解析期不可知（跨文件名字引用）；2. 两遍保证名字解析/可见性校验基于完整图，错误信息全局一致（context.go:2390-2444 的三连调用）；3. 类比 GC mark-sweep、编译器前后端分离。

- **17. GenerateAbOtaPackage 里 VABC（虚拟 A/B 压缩）为什么要求 source 与 target 双端都支持才启用?**

  参考答案要点：1. 源端 update_engine 负责写压缩/XOR 操作，目标端 snapuserd 负责在 boot 时解释；2. 任一端缺能力会导致升级后无法挂载——源码明确"disabling"（ota_from_target_files.py:1146-1167）；3. 体现 OTA 安全设计的"最差端决定能力集"。

第三级：设计题

- **18. 为你的产品线设计"debug/release 一键切换且不触发无谓全量重编"的方案。**

参考答案要点：1. 利用 katiSuffix 机制为不同配置建立不同 ninja 文件族，切换只付 kati+installclean 成本；2. 把配置差异尽量编码进 soong.productVariables 而非散落 env（env 白名单外变量不触发增量）；3. 需要彻底干净时才 DISABLE_AUTO_INSTALLCLEAN=false + installclean。

- **19. 设计一个团队级"中间产物缓存服务"：上传/下载 out/soong/.intermediates。写出正确性要点。**

参考答案要点：1. 缓存键=模块输入哈希（路径+命令行+输入内容），与 ninja restat 语义对齐；2. 下载后必须用 sbox 式"内容比较"落盘，防止 mtime 造假引发漏编；3. 复用 RBE 的缓存认证框架（rbe.go），校验产物与 .d 依赖一致性。

- **20. 若把产品配置整体迁到 rbc/Starlark，兼容性灰度策略怎么定?**

参考答案要点：1. 以 STARLARK_OPT_IN_PRODUCTS 按产品灰度（product_config.mk:231-234），双轨对拍 dump-variables-rbc；2. 保持"结果以 mk 变量文件回灌"的合同，下游 Kati/Soong 零改动；3. 迁移期跳过 ARTIFACT_PATH 重复校验（SKIP_ARTIFACT_PATH_REQUIREMENT_PRODUCTS_CHECK 分支），用 build/make/tests/ 的 rbc 用例守住回归。

术语表

术语	解释
BoardConfig.mk	板级配置：分区尺寸、签名密钥、内核路径等"硬件事实"，device 树第二层（第二章）
Blueprint	Google 的 bp 解析框架，提供 Context 流水线与 ninja writer，soong_build 建立在其上（第七章）
combined.ninja	soong_ui 用 Go template 把 soong 与 kati 的 ninja 文件按 subninja 并线后的总入口（第六章 §6.3）
ckati	Kati 的 C++ 实现，预置于 prebuilts/build-tools，负责 .mk→.ninja 翻译与 dump 变量（第九章 §9.1）
highmem_pool	combined.ninja 里的高内存动作池，限制 dex2oat 等大内存规则并发，深度由 NINJA_HIGHMEM_NUM_JOBS 覆盖（第六章 §6.3）
installclean	只删除安装产物与镜像、保留 .intermediates 的清理层级，切 lunch 时自动触发（第六章 §6.5）
katiSuffix	soong_ui 生成的 ninja 文件名后缀"-"产品名（超 64 字符退化为 md5），切 lunch 即换名（第八章 §8.6）
Microfactory	极简 Go 构建工具，绕过 bp 解析完成 soong_ui/soong_build 自举（第六章 §6.1）

术语	解释
mutator	blueprint 的图变换插件：自顶向下变体轴（arch/os/image）与自底向上依赖收集，变体爆炸的机械成因（第七章 §7.3）
previous_build_config.mk	安装目录里记录上次 product-variant 的状态文件，installCleanIfNecessary 据此判定切 lunch（第六章 §6.5）
rbc	Starlark 产品配置引擎（rbc-run），Android 13 按 STARLARK_OPT_IN_PRODUCTS 灰度，结果以 mk 文件回灌（第八章 §8.5）
restat	ninja 规则属性：重跑后输出内容未变则重置 mtime、下游不重编，增量正确性的关键（第七章 §7.5）
RuleBuilder	soong 构造 ninja 动作的现代 API：命令链 +installs+restat/sbox/highmem 声明（第七章 §7.5）
singleton	blueprint 中单实例组件（makevars/env/ninjadeps 等），注册顺序有精心安排（第七章 §7.3）
soong_ui	Go 编写的构建调度器：五阶段状态机，调度 ckati/soong_build/ninja（第六章）
sbox	soong 的沙箱执行工具：多命令原子写输出，配 --write-if-changed 消除时间戳抖动（第七章 §7.5）
stage 阶段位	RunProductConfig/RunSoong/RunKati/RunKatiNinja/RunNinja 五个位掩码，soong_ui 按位分流（第六章 §6.2）
subninja	ninja 的作用域隔离引入：被引入文件与主文件变量/规则命名空间独立（第六章 §6.3）
target-files	未签名的中间发行物 zip，按 IMAGES/SYSTEM/VENDOR/META 结构收纳产物，OTA 与镜像的共同源头（第九章 §9.3）
VABC	Virtual A/B Compression：虚拟 A/B 压缩快照，OTA 要求源/目标双端支持才启用（第九章 §9.5）
variant 变体	同一模块定义在不同轴（arch×os×link×APEX）上的克隆实例，第三章速查表与第七章 mutator 的对象
@inherit 占位符	inherit-product 写进产品变量的记账标记（INHERIT_TAG），由 _expand-inherited-values 原位展开（第八章 §8.3）

排障手册：构建系统实战问题定位

设计洞察 · 学习目标

1. 能执行"认阶段→查 error.log→回章节"的总纲流程；2. 能独立处置 ninja 子命令失败/kati 重编风暴/dex2oat OOM/签名冲突四类高频故障；3. 能说出每类故障的根因对应的机制章节并预防复发。

构建排障与运行排障（dumpsys/logcat）完全不同：错误按阶段机分层落点——阶段一报产品/变量错、阶段二报 bp 错、阶段三报 mk 错、阶段五报编译/链接/打包错。总纲：1. 认阶段（日志首行 [x/y] 前缀属于哪个阶段）→ 2. 查 out/error.log（soong_ui 专用错误流，cmd/soong_ui/main.go:199）→ 3. 回到对应章节。

场景一：ninja: build stopped: subcommand failed

症状：

text 典型报错样例（结构还原）

```
FAILED:
out/soong/.intermediates/frameworks/base/services/services/android_common/aligned/classes.ja
r
...
ninja: build stopped: subcommand failed.
23:03:51 ninja failed with: exit status 1
#### failed to build some targets (01:23 (mm:ss)) ####
```

定位命令：`grep -m1 -B2 "FAILED:" out/error.log`；真实根因通常在第一条 FAILED 之上数十行——soong_ui 屏幕只回显最后若干条，error.log 才是全量。再看该 FAILED 块内的编译命令与 stderr（如 javac 的 "error: cannot find symbol"）。

根因：ninja 的 -k 默认为 1 时遇错即收尾（runNinjaForBuild 传参 ninja.go:65-79），屏幕最后一屏几乎必然不是第一个错误——"看最后一行"是构建排障第一大坑。

修复：按 error.log 第一条 FAILED 修源码/依赖；跨模块缺符号时用 `m module-info` 后查 module-info.json 的依赖边（第一章 1.2）。

场景二：kati 重编译风暴（每次 m 都重新生成 build-<product>.ninja）

症状：每次执行 m 都卡在 `[1/1] initializing build system ...` 与 `[n/... ] including ...` 数分钟，且 `--kati_stats` 输出的变量数每次暴涨；或日志反复出现 `Build configuration changed` 之外的 kati 全量重跑。

定位命令：`ls -la out/build-<product>*.ninja`（对比 mtime 是否每次刷新）；`grep -rn '\$(shell date\|\$(shell git' device/ vendor/ build/make/core/*.mk`；对比 `cat out/soong/environment.used.*` 与当前 shell 的 env（export -p）差异。

根因：--regen 的指纹由 CLI 参数、.mk 时间戳、env 变量、$(wildcard) 与部分 $(shell) 结果组成（kati.go:93-95）。某 .mk 里写了 `$(shell date)`/`$(shell git rev-parse)` 这类每次返回不同值的表达式，或某个 env 变量每次 login 都变，都会击穿指纹。

修复：把易变值落盘成文件再 `$(file <...)` 读（main.mk 对 BUILD_NUMBER 的处理正是范本，main.mk:41-50）；修不掉的 env 用 `.KATI_READONLY`/固定值；确实无法稳定的目录排除出 wildcard。自检：连续两次 m 的第二次应在阶段三秒级通过。

场景三：dex2oat host 内存不足（OOM / Killed）

症状：

text 典型报错样例（结构还原）

```
FAILED: out/soong/.intermediates/.../dexboot/system/framework/services.jar.art
/bin/bash -c "(... dex2oatd ... ) && (...) "
/bin/bash: line 1: 12345 Killed dex2oatd ...
ninja: build stopped: subcommand failed.
```

定位命令：`dmesg | grep -i "out of memory"`（确认内核 OOM killer）；`grep -c highmem out/soong/build.ninja`；确认 combined.ninja 里 `pool highmem_pool / depth` 的值（build.go:65-67）。

根因：dex2oat 属于 highmem 动作，默认并发由内存推导（config.go:1041 起，每进程约 8GB 预算）；总并发 Parallel 默认 NumCPU+2（config.go:243），机器内存小时 highmem 动作并行数仍可能超载。

修复：`NINJA_HIGHMEM_NUM_JOBS=2 m` 限高内存池深度；或 `NINJA_ARGS="-j4" m` 限总并发（NINJA_ARGS 在 ninja.go:82-84 被追加进命令行）；长期方案是加内存或用 RBE 远端执行。

场景四：签名冲突（安装失败 / OTA 校验失败）

症状：adb install 报 `INSTALL_PARSE_FAILED_INCONSISTENT_CERTIFICATES`（覆盖安装同包名应用签名不一致）；或 OTA 升级报 `E: failed to verify whole-file signature` / recovery 里 "signature verification failed"。

定位命令：`unzip -p out/target/product/<device>/*-target_files-*.zip META/otakeys.txt`；确认构建用的是 testkey 还是 releasekey：`grep PRODUCT_DEFAULT_DEV_CERTIFICATE out/soong/make_vars-<product>.mk`；比对设备上 `dumpsys package <pkg> | grep -A2 Signatures`（需在能开机时）。

根因：构建侧默认 testkey（ota_from_target_files.py 的 -k 默认逻辑取 META/misc_info.txt 的 default_system_dev_certificate，否则 build/make/target/product/security/testkey）；设备上已装 releasekey 签名的同包名应用或同密钥 boot 校验链，混用即冲突。

修复：开发机对拍场景直接 `adb uninstall <pkg>` 后重装；发行链路用 sign_target_files_apks.py 统一重签（ReplaceOtaKeys 重写 otakeys.txt、ReplaceCerts 替换策略内证书指纹，第九章 §9.5）；OTA 包必须与设备 bootloader 内嵌密钥同源。

深度实战：构建系统实战的排障场景分析

场景：修改 framework 后运行无效果

1. m nothing 验证语法 → 通过

2. m services → 确认编译成功（BUILD SUCCESSFUL）

3. adb root && adb sync → 检查推送日志（是否推送了 services.jar）

4. kill system_server → 观察重启（书2 runtime restart）

5. 检查：可能改的是 frameworks/base/core/java（在 framework.jar 中）→ 需要 m framework

构建系统核心速查表

> **设计洞察 · 学习目标**
> 1. 用八行表格在本册内部建立"机制→锚点→章节"的最小索引；2. 能据锚点列直接跳到源码复核记忆；3. 能以此表为骨架向他人讲解本册全貌。

机制	一句话	源码锚点	所在册
Soong 四段	bp→ninja→clang/javac→image	build/soong/android/	本册
变体爆炸	一个 cc_binary 展开成 90+ 目标	Soong Go 框架	本册
增量原理	out/soong 按模块名+输入哈希缓存	ninja	本册
device 三层	chipset→device→product（mk 叠加变量）	device/google/	本册
BoardConfig	分区尺寸/签名密钥/内核路径（硬件事实）	device/<oem>/<product>/	本册
三变体	eng(root+permissive)/userdebug/root user/量产	lunch	本册
m nothing	验证 bp 语法（不产出）	soong_ui	本册
m snod	只重打 system.img（不重编模块）	Makefile	本册